

Control centralizado de intentos de validación biométrica

APP-5258 · Ampliación del alcance · 25 de agosto de 2026 **Microservicios afectados:** `onboarding-micro-person` · `facade-security` · `micro-user-audit` (consumidor) **Autor:** Deury Alcántara

1. Resumen ejecutivo

El desarrollo actual de FacePhi para Soft Token permite validar la identidad del cliente tantas veces como quiera. No hay contador, no hay bloqueo y no hay registro consolidado de intentos: cada validación es un evento aislado que sólo deja rastro en los logs de aplicación. Un atacante con la cédula de un tercero puede reintentar indefinidamente hasta que la combinación de iluminación, ángulo y calidad de imagen produzca un falso positivo.

Este documento define e implementa un **mecanismo centralizado de control de intentos**: cada validación biométrica queda registrada, los fallos consecutivos se cuentan por cliente, al quinto fallo el cliente queda bloqueado 20 minutos, y durante el bloqueo el microservicio ni siquiera invoca a FacePhi. Todo el ciclo se audita en `audit-log`, y el bloqueo se comunica a la app móvil con un **HTTP 423 Locked** que incluye cuándo podrá reintentar.

Tres reglas gobiernan todo el diseño:

1. **La lógica vive sólo en `onboarding-micro-person`**. El facade propaga; no cuenta, no bloquea, no decide. Es la misma regla que ya se aplicó al resto del desarrollo de FacePhi.
2. **Un error técnico no es un intento fallido**. Si FacePhi está caído, el cliente no puede quedar bloqueado. Esto es lo único del requerimiento que el código actual **no puede cumplir sin un cambio previo**, y se detalla en la sección 11.
3. **El estado del Soft Token no se toca**. Ni en rechazos, ni en errores, ni en bloqueos.

Estimación: 15 puntos, repartidos en 6 subtarefas (sección 14).

2. Qué pide el requerimiento

Transcripción estructurada de lo solicitado, para que quede trazable qué se implementa y por qué.

2.1 Objetivo

Se requiere implementar un mecanismo para registrar, controlar y bloquear temporalmente los intentos fallidos de validación biométrica con FacePhi.

2.2 Reglas de negocio

#	Regla	Dónde se implementa
R1	Registrar cada intento de validación y su resultado	<code>BiometricAttemptControlService</code> + <code>audit-log</code>
R2	Contador de fallos consecutivos por cliente	Colección <code>facephi_attempt_control</code>
R3	Bloqueo al alcanzar 5 fallos consecutivos	<code>RegisterFailedAttemptAsync</code> (paso 2, condicional)
R4	Bloqueo temporal de 20 minutos	<code>BlockedUntil = ahora +</code> <code>BIOMETRIC_BLOCK_DURATION_MINUTES</code>
R5	Durante el bloqueo no se invoca a FacePhi	<code>BiometricAttemptControlBehaviour</code> (pipeline MediatR)
R6	Reiniciar el contador tras un éxito o al expirar el bloqueo	<code>RegisterSuccessfulAttemptAsync</code> / <code>ReleaseExpiredBlockAsync</code>
R7	El estado del Soft Token permanece sin cambios ante rechazos, errores técnicos o bloqueos	No se añade ninguna dependencia hacia Soft Token

2.3 Auditoría

Debe usarse la integración existente con `audit-log` en `micro-user-audit`. Eventos mínimos:

- Intentos exitosos
- Intentos fallidos
- Bloqueo por exceso de intentos
- Intentos realizados durante el bloqueo
- Desbloqueo automático
- Errores técnicos

Campos mínimos: cliente, fecha, flujo, resultado, contador, estado de bloqueo y trazabilidad.

2.4 Frontend

Se requiere una respuesta explícita para que **Móvil APAP** identifique el bloqueo: **HTTP 423**.

2.5 Reparto por servicio

Servicio	Responsabilidad
<code>onboarding-micro-person</code>	Contador y estado de bloqueo · validar bloqueo antes de invocar FacePhi · registrar eventos en <code>audit-log</code> · reiniciar/incrementar según resultado · devolver el contrato de bloqueo
<code>facade-security</code>	Propagar la respuesta · conservar el 423 · documentarlo en Swagger · no implementar ni duplicar la lógica de intentos

2.6 Persistencia

Dos fuentes separadas. El estado guarda: `idT24`, intentos fallidos consecutivos, estado activo/bloqueado, fecha de inicio del bloqueo, fecha de fin del bloqueo, último flujo ejecutado y fecha del último intento.

2.7 Configuración

Variable	Valor
BIOMETRIC_MAX_FAILED_ATTEMPTS	5
BIOMETRIC_BLOCK_DURATION_MINUTES	20
BIOMETRIC_ATTEMPT_CONTROL_ENABLED	true

2.8 Pruebas exigidas

Nueve escenarios, listados y mapeados a pruebas concretas en la sección 12.

3. Decisiones de diseño

Tema	Decisión	Por qué
Dónde se engancha el control	Behaviour del pipeline de MediatR , no código dentro de cada handler	Los dos flujos (<code>validate</code> y <code>validate-face</code>) necesitan exactamente el mismo control. Duplicarlo garantiza que algún día uno de los dos se quede desactualizado. Con el behaviour, la regla "no se llama a FacePhi estando bloqueado" es estructural: el handler ni siquiera se ejecuta
Persistencia del estado	Colección Mongo nueva y separada , <code>facephi_attempt_control</code>	Mezclarla con <code>facephi_biometrics</code> obligaría a escribir en el documento que guarda <code>token1</code> en cada intento fallido, justo el registro que no se quiere tocar
Incremento del contador	<code>\$inc</code> atómico del lado del servidor (<code>FindOneAndUpdateAsync</code>)	Con leer-modificar-escribir, dos peticiones concurrentes leen 4, escriben 5 las dos, y el cliente gana un intento gratis en cada carrera. Un contador de seguridad no se puede implementar así
Aplicación del bloqueo	Segunda operación condicional (<code>isBlocked == false</code>)	Si dos peticiones cruzan el umbral a la vez, sólo una escribe la ventana. La otra no reinicia el reloj del bloqueo
Desbloqueo automático	Perezoso : se evalúa en el momento del siguiente intento	Un job programado añade un componente más, depende del reloj de cada pod y no cambia nada de lo que ve el cliente: el bloqueo sólo tiene efecto cuando alguien intenta validar
Ámbito del contador	Por cliente (<code>idT24</code>), compartido entre los dos flujos	Si fuera por endpoint, bastaría con alternar <code>validate</code> y <code>validate-face</code> para duplicar los intentos disponibles
Error técnico	Se audita, no incrementa	Exigido por el requerimiento. Además: si contara, una caída de FacePhi bloquearía a todos los clientes del país en cinco minutos
Petición inválida (400) y "sin documento" (404)	No consumen intento	En ninguno de los dos casos se llegó a invocar a FacePhi. No hay intento biométrico que contar
Intento durante el bloqueo	Se audita, no incrementa, no alarga el bloqueo	De lo contrario, martillear el endpoint mantendría al cliente bloqueado indefinidamente
Fallo al auditar	Se registra como error y el flujo continúa	La auditoría no puede tumbar la validación biométrica de un cliente. La fuente de verdad del contador es Mongo, no <code>audit-log</code>
Reloj	<code>TimeProvider</code> (BCL de .NET 8)	Permite probar la expiración del bloqueo sin esperar 20 minutos y sin añadir paquetes
Intentos restantes en la respuesta	No se exponen	El requerimiento no los pide. Queda como pregunta abierta para UX (sección 16)

4. Arquitectura

4.1 Ubicación de las piezas

```
onboarding-micro-person
|
├─ Endpoints/FacePhi.cs                (sin cambios funcionales)
|
├─ Application/
|   └─ Common/Behaviours/
|       └─ ValidationBehaviour.cs        ← ya existe
|       └─ BiometricAttemptControlBehaviour.cs ← NUEVO (se registra DESPUÉS de ValidationBehaviour)
|       └─ PerformanceBehaviour.cs      ← ya existe
|       └─ UnhandledExceptionBehaviour.cs ← ya existe
|   └─ Facephi/Commands/
|       └─ ValidateIdentityV2Cmd.cs      ← MODIFICADO (implementa IBiometricAttemptControlled)
|       └─ ValidateFaceOnlyCmd.cs       ← MODIFICADO (implementa IBiometricAttemptControlled)
|
├─ Infrastructure/Biometric/
|   └─ BiometricAttemptControlService.cs ← NUEVO (toda la política)
|   └─ BiometricAuditPublisher.cs       ← NUEVO (envoltorio de audit-log)
|
├─ Persistence/
|   └─ FacePhiAttemptControlRepository.cs ← NUEVO
|   └─ FacePhiAttemptControlIndexInitializer.cs ← NUEVO
|
└─ Common/
    └─ Entities/FacePhiAttemptControlDocument.cs ← NUEVO
    └─ Enums/Biometric/BiometricFlow.cs         ← NUEVO
    └─ Models/Biometric/BiometricAuditEvent.cs  ← NUEVO
    └─ Interfaces/Biometric/
        └─ IFacePhiAttemptControlRepository.cs ← NUEVO
        └─ IBiometricAttemptControlled.cs      ← NUEVO
    └─ Exceptions/BiometricAttemptsBlockedException.cs ← NUEVO
    └─ Exceptions/Handlers/BiometricAttemptsBlockedExceptionHandler.cs ← NUEVO
    └─ Configuration/AppSettings.cs            ← MODIFICADO (3 variables)

facade-security
└─ src/modules/facephi/
    └─ facephi.controller.ts                ← MODIFICADO (sólo Swagger: @ApiResponse 423)
    └─ facephi.service.ts                   ← SIN CAMBIOS
    └─ facephi.service.spec.ts              ← MODIFICADO (pruebas de propagación del 423)
```

4.2 Flujo completo



5. Modelo de datos

5.1 Las dos fuentes de persistencia

El requerimiento pide dos fuentes separadas. Son éstas:

Fuente	Naturaleza	Contenido	Dónde vive
Estado operativo	Mutable, un documento por cliente	Contador vigente y ventana de bloqueo	Mongo, colección <code>facephi_attempt_control</code> , en <code>onboarding-micro-person</code>
Auditoría	Inmutable, un registro por evento	Histórico completo de lo ocurrido	<code>audit-log</code> , consumido por <code>micro-user-audit</code>

La distinción importa: el estado responde "¿puede este cliente intentar ahora?" y se sobrescribe; la auditoría responde "¿qué pasó el martes a las 3?" y no se sobrescribe nunca. Si sólo existiera el estado, un bloqueo liberado sería indistinguible de un bloqueo que nunca ocurrió.

5.2 Colección `facephi_attempt_control`

```
{
  "_id": "66c9f1e2a4b3c2d1e0f9a8b7",
  "idT24": "123456789",
  "failedAttempts": 5,
  "isBlocked": true,
  "blockedAt": "2026-08-25T14:00:00Z",
  "blockedUntil": "2026-08-25T14:20:00Z",
  "lastFlow": "validate-face",
  "lastAttemptAt": "2026-08-25T14:00:00Z",
  "createdAt": "2026-08-20T09:11:03Z",
  "updatedAt": "2026-08-25T14:00:00Z"
}
```

Ningún campo biométrico entra aquí. Esta colección no guarda `token1`, `token2`, `bestImageToken` ni `extraData`; si alguien la volcara completa a un CSV, no habría fuga de datos biométricos.

5.3 Índices

Índice	Definición	Para qué
<code>ux_idT24</code>	<code>{ idT24: 1 }</code> , único	Sin él, dos peticiones concurrentes con <code>upsert</code> pueden insertar dos documentos para el mismo cliente, y a partir de ahí cada uno lleva su propio contador — el cliente tendría 10 intentos en lugar de 5
<code>ix_isBlocked_blockedUntil</code>	<code>{ isBlocked: 1, blockedUntil: 1 }</code>	Soporte y monitoreo: "¿cuántos clientes están bloqueados ahora mismo?"

Aprovechar el mismo cambio: la colección `facephi_biometrics` sigue **sin índice único sobre `idT24`**. Es la misma vulnerabilidad de carrera, observada en el `ReplaceOneAsync` con `IsUpsert` del `FacePhiBiometricRepository`. Conviene crear los dos índices en el mismo despliegue.

5.4 Retención

El documento de estado **no lleva TTL**. Es un registro operativo pequeño (menos de 300 bytes por cliente) y su antigüedad es precisamente lo que permite reconstruir el comportamiento. La retención regulatoria la cubre `audit-log`, según la política que ya aplique `micro-user-audit`.

6. Contrato del bloqueo (HTTP 423)

6.1 Respuesta del microservicio

HTTP/1.1 423 Locked

Content-Type: application/problem+json

Retry-After: 1200

```
{
  "type": "https://tools.ietf.org/html/rfc4918#section-11.3",
  "title": "Biometric validation temporarily blocked",
  "status": 423,
  "detail": "The customer exceeded the allowed number of failed biometric attempts.",
  "instance": "/api/v1/facephi/validate-face",
  "code": "BIOMETRIC_ATTEMPTS_BLOCKED",
  "isValid": false,
  "blocked": true,
  "blockedUntil": "2026-08-25T14:20:00Z",
  "retryAfterSeconds": 1200
}
```

Tres detalles deliberados:

- `isValid`: `false` **también en el 423**. La app lee siempre el mismo campo, responda 200 o 423. No necesita dos rutas de parseo distintas.
- `retryAfterSeconds` **además de** `blockedUntil` . Un temporizador en el móvil hecho a partir de `blockedUntil` depende de que el reloj del teléfono esté sincronizado; con los segundos restantes, no depende de nada.
- **Cabecera** `Retry-After` **estándar**. Además del cuerpo, para que cualquier cliente HTTP genérico (un gateway, un reintento automático) sepa comportarse sin conocer nuestro contrato.

6.2 Tabla completa de respuestas

Situación	Status	Cuerpo	¿Consume intento?
Validación aprobada	200	{ "isValid": true }	Sí — reinicia el contador a 0
Validación rechazada	200	{ "isValid": false }	Sí — incrementa el contador
Petición inválida	400	ProblemDetails	No
Cliente sin documento almacenado	404	ProblemDetails	No
Cliente bloqueado	423	ProblemDetails + <code>blockedUntil</code>	No — ni siquiera se llama a FacePhi
FacePhi no disponible	502	ProblemDetails	No — se audita como error técnico
Timeout hacia FacePhi	504	ProblemDetails	No — se audita como error técnico

6.3 Qué debe hacer Móvil APAP

Ante un `423` : mostrar la pantalla de bloqueo con la cuenta atrás de `retryAfterSeconds` y **deshabilitar el botón de reintentar** hasta que expire. Volver a llamar durante el bloqueo devuelve otro `423` , genera un

evento de auditoría más y no adelanta el desbloqueo.

Ante un `502 / 504` : mensaje de servicio no disponible, con reintento permitido. No es culpa del cliente y no gastó ningún intento.

7. Implementación · `onboarding-micro-person`

7.1 Entidad de estado

Archivo nuevo: `src/micro-person-api/Common/Entities/FacePhiAttemptControlDocument.cs`

```

// Ruta destino:
// src/micro-person-api/Common/Entities/FacePhiAttemptControlDocument.cs
//
// Colección Mongo: "facephi_attempt_control".
// Es una colección NUEVA y SEPARADA de "facephi_biometrics": aquélla guarda
// datos biométricos (token1/token2), ésta guarda únicamente estado de control.
// Mezclarlas obligaría a escribir en el documento biométrico en cada intento
// fallido, que es justo el escenario en el que no se quiere tocar.

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace onboarding_micro_person.Common.Entities
{
    public class FacePhiAttemptControlDocument
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string? Id { get; set; }

        /// <summary>Identificador del cliente en T24. Clave de negocio, índice único.</summary>
        [BsonElement("idT24")]
        public string IdT24 { get; set; } = string.Empty;

        /// <summary>Intentos biométricos fallidos consecutivos. Se reinicia en 0 tras un éxito o tras expirar
        el bloqueo.</summary>
        [BsonElement("failedAttempts")]
        public int FailedAttempts { get; set; }

        /// <summary>Estado de bloqueo vigente al momento de la última escritura.</summary>
        [BsonElement("isBlocked")]
        public bool IsBlocked { get; set; }

        /// <summary>Instante en que se aplicó el bloqueo (UTC). Null si nunca se bloqueó o si ya se liberó.
        </summary>
        [BsonElement("blockedAt")]
        public DateTime? BlockedAt { get; set; }

        /// <summary>Instante en que el bloqueo deja de tener efecto (UTC).</summary>
        [BsonElement("blockedUntil")]
        public DateTime? BlockedUntil { get; set; }

        /// <summary>Último flujo ejecutado: "validate-biometric" o "validate-face".</summary>
        [BsonElement("lastFlow")]
        public string? LastFlow { get; set; }

        /// <summary>Fecha del último intento registrado (UTC).</summary>
        [BsonElement("lastAttemptAt")]
        public DateTime? LastAttemptAt { get; set; }
    }
}

```

```

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; }

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; }

/// <summary>
/// El bloqueo está vigente sólo si la bandera está activa Y la ventana no ha expirado.
/// La bandera por sí sola no basta: nadie la apaga hasta el siguiente intento.
/// </summary>
public bool IsCurrentlyBlocked(DateTime utcNow) =>
    IsBlocked && BlockedUntil is not null && BlockedUntil > utcNow;

/// <summary>Segundos que faltan para poder reintentar. Nunca negativo.</summary>
public int RemainingSeconds(DateTime utcNow)
{
    if (BlockedUntil is null)
    {
        return 0;
    }

    var remaining = (BlockedUntil.Value - utcNow).TotalSeconds;

    return remaining <= 0 ? 0 : (int)Math.Ceiling(remaining);
}
}

```

7.2 Enum de flujo

Archivo nuevo: src/micro-person-api/Common/Enums/Biometric/BiometricFlow.cs

```
// Ruta destino:
// src/micro-person-api/Common/Enums/Biometric/BiometricFlow.cs
//
// Sigue la convención de FacePhiTrackingReason: enum + extensión ToApiString(),
// para que el valor que viaja a Mongo y a audit-log sea una cadena estable y no
// el número del enum (renumerar el enum no debe cambiar el histórico).

namespace onboarding_micro_person.Common.Enums.Biometric
{
    public enum BiometricFlow
    {
        /// <summary>POST /api/v1/facephi/validate – documento capturado + selfie.</summary>
        ValidateBiometric = 1,

        /// <summary>POST /api/v1/facephi/validate-face – selfie contra el documento ya almacenado.</summary>
        ValidateFace = 2
    }

    public static class BiometricFlowExtensions
    {
        public static string ToApiString(this BiometricFlow flow) => flow switch
        {
            BiometricFlow.ValidateBiometric => "validate-biometric",
            BiometricFlow.ValidateFace => "validate-face",
            _ => "unknown"
        };
    }
}
```

7.3 Contrato del repositorio

Archivo nuevo: src/micro-person-api/Common/Interfaces/Biometric/IFacePhiAttemptControlRepository.cs

```

// Ruta destino:
// src/micro-person-api/Common/Interfaces/Biometric/IFacePhiAttemptControlRepository.cs

using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Enums.Biometric;

namespace onboarding_micro_person.Common.Interfaces.Biometric
{
    public interface IFacePhiAttemptControlRepository
    {
        /// <summary>Lee el estado de control del cliente. Null si nunca ha tenido intentos.</summary>
        Task<FacePhiAttemptControlDocument?> GetByIdT24Async(
            string idT24,
            CancellationToken cancellationToken);

        /// <summary>
        /// Incrementa el contador de fallos de forma atómica y, si se alcanza el umbral,
        /// aplica el bloqueo. Devuelve el estado resultante.
        /// </summary>
        Task<FacePhiAttemptControlDocument> RegisterFailedAttemptAsync(
            string idT24,
            BiometricFlow flow,
            int maxFailedAttempts,
            int blockDurationMinutes,
            DateTime utcNow,
            CancellationToken cancellationToken);

        /// <summary>Reinicia el contador y limpia el bloqueo tras un intento exitoso.</summary>
        Task<FacePhiAttemptControlDocument> RegisterSuccessfulAttemptAsync(
            string idT24,
            BiometricFlow flow,
            DateTime utcNow,
            CancellationToken cancellationToken);

        /// <summary>
        /// Libera un bloqueo cuya ventana ya expiró y reinicia el contador.
        /// Condicional: sólo actúa si el documento sigue bloqueado y BlockedUntil ya pasó,
        /// para que dos peticiones simultáneas no produzcan dos desbloques.
        /// Devuelve null si otra petición se adelantó.
        /// </summary>
        Task<FacePhiAttemptControlDocument?> ReleaseExpiredBlockAsync(
            string idT24,
            DateTime utcNow,
            CancellationToken cancellationToken);

        /// <summary>
        /// Deja constancia de un intento que no altera el contador (error técnico).
        /// Sólo actualiza lastFlow / lastAttemptAt / updatedAt.
        /// </summary>

```

```
Task TouchAttemptAsync(  
    string idT24,  
    BiometricFlow flow,  
    DateTime utcNow,  
    CancellationToken cancellationToken);  
}  
}
```

7.4 Repositorio

Archivo nuevo: `src/micro-person-api/Persistence/FacePhiAttemptControlRepository.cs`

Es la pieza más delicada del desarrollo. Un contador de seguridad implementado con leer-modificar-escribir es un contador que se puede burlar abriendo dos peticiones a la vez, así que todo el incremento ocurre del lado del servidor de Mongo.

```

// Ruta destino:
// src/micro-person-api/Persistence/FacePhiAttemptControlRepository.cs
//
// Ajusta el using de la conexión Mongo al que ya usa FacePhiBiometricRepository
// (ahí se resuelve el IMongoDatabase). Este repositorio NO usa ReplaceOneAsync:
// un contador de seguridad se incrementa con $inc del lado del servidor, nunca
// leyendo-modificando-escribiendo desde la aplicación. Con read-modify-write dos
// peticiones concurrentes leen 4, escriben 5 las dos, y el cliente consigue un
// intento gratis en cada carrera.

using MongoDB.Driver;
using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Enums.Biometric;
using onboarding_micro_person.Common.Interfaces.Biometric;

namespace onboarding_micro_person.Persistence
{
    public class FacePhiAttemptControlRepository : IFacePhiAttemptControlRepository
    {
        public const string CollectionName = "facephi_attempt_control";

        private readonly IMongoCollection<FacePhiAttemptControlDocument> _collection;

        private static readonly FilterDefinitionBuilder<FacePhiAttemptControlDocument> Filter =
            Builders<FacePhiAttemptControlDocument>.Filter;

        private static readonly UpdateDefinitionBuilder<FacePhiAttemptControlDocument> Update =
            Builders<FacePhiAttemptControlDocument>.Update;

        public FacePhiAttemptControlRepository(IMongoDatabase database)
        {
            _collection = database.GetCollection<FacePhiAttemptControlDocument>(CollectionName);
        }

        public async Task<FacePhiAttemptControlDocument?> GetByIdT24Async(
            string idT24,
            CancellationToken cancellationToken)
        {
            return await _collection
                .Find(d => d.IdT24 == idT24)
                .FirstOrDefaultAsync(cancellationToken);
        }

        public async Task<FacePhiAttemptControlDocument> RegisterFailedAttemptAsync(
            string idT24,
            BiometricFlow flow,
            int maxFailedAttempts,
            int blockDurationMinutes,
            DateTime utcNow,

```

```

Cancellation token cancellationToken)
{
    // Paso 1: incremento atómico. El upsert crea el documento la primera vez;
    // el idT24 lo siembra Mongo a partir de la igualdad del filtro.
    var increment = Update
        .Inc(d => d.FailedAttempts, 1)
        .Set(d => d.LastFlow, flow.ToApiString())
        .Set(d => d.LastAttemptAt, utcNow)
        .Set(d => d.UpdatedAt, utcNow)
        .SetOnInsert(d => d.IsBlocked, false)
        .SetOnInsert(d => d.CreatedAt, utcNow);

    var state = await _collection.FindOneAndUpdateAsync(
        Filter.Eq(d => d.IdT24, idT24),
        increment,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            IsUpsert = true,
            ReturnDocument = ReturnDocument.After
        },
        cancellationToken);

    if (state.FailedAttempts < maxFailedAttempts || state.IsBlocked)
    {
        return state;
    }

    // Paso 2: bloqueo condicional. El filtro exige isBlocked == false, así que
    // si dos peticiones cruzan el umbral a la vez sólo una escribe la ventana
    // y la otra recibe null (no reinicia el reloj del bloqueo).
    var blockUpdate = Update
        .Set(d => d.IsBlocked, true)
        .Set(d => d.BlockedAt, utcNow)
        .Set(d => d.BlockedUntil, utcNow.AddMinutes(blockDurationMinutes))
        .Set(d => d.UpdatedAt, utcNow);

    var blocked = await _collection.FindOneAndUpdateAsync(
        Filter.And(
            Filter.Eq(d => d.IdT24, idT24),
            Filter.Eq(d => d.IsBlocked, false)),
        blockUpdate,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            ReturnDocument = ReturnDocument.After
        },
        cancellationToken);

    return blocked ?? state;
}

```



```

public async Task<FacePhiAttemptControlDocument> RegisterSuccessfulAttemptAsync(
    string idT24,
    BiometricFlow flow,
    DateTime utcNow,
    Cancellation_token cancellationToken)
{
    var update = Update
        .Set(d => d.FailedAttempts, 0)
        .Set(d => d.IsBlocked, false)
        .Set(d => d.BlockedAt, (DateTime?)null)
        .Set(d => d.BlockedUntil, (DateTime?)null)
        .Set(d => d.LastFlow, flow.ToApiString())
        .Set(d => d.LastAttemptAt, utcNow)
        .Set(d => d.UpdatedAt, utcNow)
        .SetOnInsert(d => d.CreatedAt, utcNow);

    return await _collection.FindOneAndUpdateAsync(
        Filter.Eq(d => d.IdT24, idT24),
        update,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            IsUpsert = true,
            ReturnDocument = ReturnDocument.After
        },
        cancellationToken);
}

```

```

public async Task<FacePhiAttemptControlDocument?> ReleaseExpiredBlockAsync(
    string idT24,
    DateTime utcNow,
    Cancellation_token cancellationToken)
{
    var update = Update
        .Set(d => d.IsBlocked, false)
        .Set(d => d.FailedAttempts, 0)
        .Set(d => d.BlockedAt, (DateTime?)null)
        .Set(d => d.BlockedUntil, (DateTime?)null)
        .Set(d => d.UpdatedAt, utcNow);

    return await _collection.FindOneAndUpdateAsync(
        Filter.And(
            Filter.Eq(d => d.IdT24, idT24),
            Filter.Eq(d => d.IsBlocked, true),
            Filter.Lte(d => d.BlockedUntil, utcNow)),
        update,
        new FindOneAndUpdateOptions<FacePhiAttemptControlDocument>
        {
            ReturnDocument = ReturnDocument.After
        }
    );
}

```

```

        },
        cancellationTokens);
    }

    public async Task TouchAttemptAsync(
        string idT24,
        BiometricFlow flow,
        DateTime utcNow,
        CancellationToken cancellationToken)
    {
        var update = Update
            .Set(d => d.LastFlow, flow.ToApiString())
            .Set(d => d.LastAttemptAt, utcNow)
            .Set(d => d.UpdatedAt, utcNow)
            .SetOnInsert(d => d.FailedAttempts, 0)
            .SetOnInsert(d => d.IsBlocked, false)
            .SetOnInsert(d => d.CreatedAt, utcNow);

        await _collection.UpdateOneAsync(
            Filter.Eq(d => d.IdT24, idT24),
            update,
            new UpdateOptions { IsUpsert = true },
            cancellationToken);
    }
}

```

7.5 Índices

Archivo nuevo: `src/micro-person-api/Persistence/FacePhiAttemptControlIndexInitializer.cs`

```
// Ruta destino:
// src/micro-person-api/Persistence/FacePhiAttemptControlIndexInitializer.cs
//
// El índice único sobre idT24 no es decorativo: sin él, dos peticiones
// concurrentes con upsert pueden insertar DOS documentos para el mismo cliente,
// y a partir de ahí cada uno lleva su propio contador. Con el índice único, la
// segunda inserción falla y Mongo reintenta el update sobre el documento que ya
// existe, que es el comportamiento que queremos.
//
// Registro en Program.cs: builder.Services.AddHostedService<FacePhiAttemptControlIndexInitializer>();
//
// Si el micro ya tiene un inicializador de índices para facephi_biometrics,
// añade aquí el índice en lugar de crear una clase nueva (y aprovecha para
// crear también el índice único de idT24 en facephi_biometrics, que sigue
// pendiente).

using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using MongoDB.Driver;
using onboarding_micro_person.Common.Entities;

namespace onboarding_micro_person.Persistence
{
    public class FacePhiAttemptControlIndexInitializer(
        IMongoDatabase database,
        ILogger<FacePhiAttemptControlIndexInitializer> logger) : IHostedService
    {
        private readonly IMongoDatabase _database = database;
        private readonly ILogger<FacePhiAttemptControlIndexInitializer> _logger = logger;

        public async Task StartAsync(CancellationToken cancellationToken)
        {
            try
            {
                var collection = _database.GetCollection<FacePhiAttemptControlDocument>(
                    FacePhiAttemptControlRepository.CollectionName);

                var uniqueIdT24 = new CreateIndexModel<FacePhiAttemptControlDocument>(
                    Builders<FacePhiAttemptControlDocument>.IndexKeys.Ascending(d => d.IdT24),
                    new CreateIndexOptions { Unique = true, Name = "ux_idT24" });

                // Soporte y monitoreo: "¿cuántos clientes están bloqueados ahora mismo?"
                var blockedUntil = new CreateIndexModel<FacePhiAttemptControlDocument>(
                    Builders<FacePhiAttemptControlDocument>.IndexKeys
                        .Ascending(d => d.IsBlocked)
                        .Ascending(d => d.BlockedUntil),
                    new CreateIndexOptions { Name = "ix_isBlocked_blockedUntil" });

                await collection.Indexes.CreateManyAsync(
```

```

        new[] { uniqueIdT24, blockedUntil },
        cancellationToken);

        _logger.LogInformation(
            "Indexes ensured for collection {Collection}",
            FacePhiAttemptControlRepository.CollectionName);
    }
    catch (Exception ex)
    {
        // Un fallo creando índices no debe impedir el arranque del micro,
        // pero tiene que quedar visible: sin el índice único el control
        // de intentos es vulnerable a la carrera de inserción.
        _logger.LogError(
            ex,
            "Could not ensure indexes for collection {Collection}",
            FacePhiAttemptControlRepository.CollectionName);
    }
}

public Task StopAsync(Cancellation_token cancellationToken) => Task.CompletedTask;
}
}

```

7.6 Excepción de bloqueo

Archivo nuevo: src/micro-person-api/Common/Exceptions/BiometricAttemptsBlockedException.cs

```
// Ruta destino:
// src/micro-person-api/Common/Exceptions/BiometricAttemptsBlockedException.cs
//
// Se traduce a HTTP 423 Locked en el IExceptionHandler del micro.
// No hereda de ninguna excepción de validación ni de negocio existente a
// propósito: el pipeline de control de intentos la deja pasar sin tocar el
// contador, y confundirla con otra familia rompería esa regla.

namespace onboarding_micro_person.Common.Exceptions
{
    public class BiometricAttemptsBlockedException : Exception
    {
        public const string ErrorCode = "BIOMETRIC_ATTEMPTS_BLOCKED";

        public BiometricAttemptsBlockedException(
            string idT24,
            DateTime blockedUntil,
            int retryAfterSeconds)
            : base("Biometric validation is temporarily blocked for this customer.")
        {
            IdT24 = idT24;
            BlockedUntil = blockedUntil;
            RetryAfterSeconds = retryAfterSeconds;
        }

        public string IdT24 { get; }

        /// <summary>Instante UTC en el que el cliente puede volver a intentar.</summary>
        public DateTime BlockedUntil { get; }

        /// <summary>Segundos restantes. Viaja en la cabecera Retry-After y en el cuerpo.</summary>
        public int RetryAfterSeconds { get; }
    }
}
```

7.7 Modelo del evento de auditoría

Archivo nuevo: src/micro-person-api/Common/Models/Biometric/BiometricAuditEvent.cs

```

// Ruta destino:
// src/micro-person-api/Common/Models/Biometric/BiometricAuditEvent.cs
//
// Modelo del evento que se envía a audit-log (micro-user-audit).
// Regla dura: aquí NO entra ningún dato biométrico. Nada de token1, token2,
// bestImageToken ni extraData. Lo que se audita es el hecho y su trazabilidad,
// no la evidencia.

using onboarding_micro_person.Common.Enums.Biometric;

namespace onboarding_micro_person.Common.Models.Biometric
{
    /// <summary>Catálogo cerrado de eventos auditables del control de intentos.</summary>
    public enum BiometricAuditEventType
    {
        AttemptSuccess = 1,
        AttemptFailed = 2,
        BlockedByExcess = 3,
        AttemptWhileBlocked = 4,
        AutomaticUnblock = 5,
        TechnicalError = 6
    }

    public static class BiometricAuditEventTypeExtensions
    {
        public static string ToApiString(this BiometricAuditEventType type) => type switch
        {
            BiometricAuditEventType.AttemptSuccess => "BIOMETRIC_ATTEMPT_SUCCESS",
            BiometricAuditEventType.AttemptFailed => "BIOMETRIC_ATTEMPT_FAILED",
            BiometricAuditEventType.BlockedByExcess => "BIOMETRIC_BLOCKED_BY_EXCESS",
            BiometricAuditEventType.AttemptWhileBlocked => "BIOMETRIC_ATTEMPT_WHILE_BLOCKED",
            BiometricAuditEventType.AutomaticUnblock => "BIOMETRIC_AUTOMATIC_UNBLOCK",
            BiometricAuditEventType.TechnicalError => "BIOMETRIC_TECHNICAL_ERROR",
            _ => "BIOMETRIC_UNKNOWN"
        };
    }

    public class BiometricAuditEvent
    {
        public BiometricAuditEventType Type { get; set; }

        /// <summary>Cliente.</summary>
        public string IdT24 { get; set; } = string.Empty;

        /// <summary>Fecha del evento en UTC.</summary>
        public DateTime OccurredAt { get; set; }

        /// <summary>Flujo: "validate-biometric" o "validate-face".</summary>
        public string Flow { get; set; } = string.Empty;
    }
}

```

```

    /// <summary>Resultado legible: APPROVED | REJECTED | BLOCKED | ERROR | UNBLOCKED.</summary>
    public string Result { get; set; } = string.Empty;

    /// <summary>Contador de fallos consecutivos después de aplicar este evento.</summary>
    public int FailedAttempts { get; set; }

    /// <summary>Estado de bloqueo después de aplicar este evento.</summary>
    public bool Blocked { get; set; }

    public DateTime? BlockedUntil { get; set; }

    /// <summary>requestId de la petición, para cruzar con los logs del micro y del facade.</summary>
    public string? RequestId { get; set; }

    /// <summary>operationId del tracking de FacePhi, cuando la petición lo trae.</summary>
    public string? OperationId { get; set; }

    /// <summary>serviceTransactionId devuelto por FacePhi, cuando llegó a responder.</summary>
    public string? ServiceTransactionId { get; set; }

    /// <summary>Sólo para TechnicalError: tipo de la excepción o código de servicio. Sin stack trace.
    </summary>
    public string? ErrorDetail { get; set; }

    public static BiometricAuditEvent For(
        BiometricAuditEventType type,
        string idT24,
        BiometricFlow flow,
        DateTime utcNow,
        string result) => new()
    {
        Type = type,
        IdT24 = idT24,
        Flow = flow.ToApiString(),
        OccurredAt = utcNow,
        Result = result
    };
}
}

```

7.8 Publicador de auditoría

Archivo nuevo: `src/micro-person-api/Infrastructure/Biometric/BiometricAuditPublisher.cs`

Requiere descubrimiento en el repositorio. El micro ya tiene integración con `audit-log`; hay que localizar el cliente real y ajustar la firma. Los comandos de búsqueda están en la cabecera del archivo y en la sección 15.

```
// Ruta destino:
// src/micro-person-api/Infrastructure/Biometric/BiometricAuditPublisher.cs
//
// La interfaz va en el mismo archivo; muévela a Common/Interfaces/Biometric/
// si el micro separa contratos.
//
// — ANTES DE USAR ESTE ARCHIVO —
// El micro YA tiene integración con audit-log (micro-user-audit). Localízala:
//
// grep -rn "audit" --include=*.cs src/ -il
// grep -rn "IAuditLog\|AuditService\|AuditClient\|audit-log" --include=*.cs src/
// grep -rni "audit" src/micro-person-api/appsettings*.json
//
// Sustituye IAuditLogService / RegisterAsync por el cliente y la firma reales.
// Lo único que NO debe cambiar es el envoltorio: publicación no bloqueante y
// errores tragados, por el motivo que se explica abajo.
// —————

using Microsoft.Extensions.Logging;
using onboarding_micro_person.Common.Interfaces.Biometric;
using onboarding_micro_person.Common.Models.Biometric;

namespace onboarding_micro_person.Infrastructure.Biometric
{
    public interface IBiometricAuditPublisher
    {
        Task PublishAsync(BiometricAuditEvent auditEvent, CancellationToken cancellationToken);
    }

    public class BiometricAuditPublisher(
        IAuditLogService auditLog,
        ILogger<BiometricAuditPublisher> logger) : IBiometricAuditPublisher
    {
        private readonly IAuditLogService _auditLog = auditLog;
        private readonly ILogger<BiometricAuditPublisher> _logger = logger;

        public async Task PublishAsync(
            BiometricAuditEvent auditEvent,
            CancellationToken cancellationToken)
        {
            try
            {
                // Adapta este payload al contrato real de audit-log.
                await _auditLog.RegisterAsync(new
                {
                    eventType = auditEvent.Type.ToApiString(),
                    idT24 = auditEvent.IdT24,
                    occurredAt = auditEvent.OccurredAt,
                    flow = auditEvent.Flow,
                })
            }
            catch { }
        }
    }
}
```



```

        result = auditEvent.Result,
        failedAttempts = auditEvent.FailedAttempts,
        blocked = auditEvent.Blocked,
        blockedUntil = auditEvent.BlockedUntil,
        requestId = auditEvent.RequestId,
        operationId = auditEvent.OperationId,
        serviceTransactionId = auditEvent.ServiceTransactionId,
        errorDetail = auditEvent.ErrorDetail
    }, cancellationToken);
}
catch (Exception ex)
{
    // Decisión consciente: la auditoría no puede tumbar la validación
    // biométrica de un cliente. Si audit-log está caído, el cliente
    // sigue pudiendo activar su token y el contador sigue siendo
    // correcto, porque la fuente de verdad del contador es Mongo, no
    // la auditoría. Queda el error en los logs del micro para que la
    // pérdida de eventos sea detectable y alertable.
    _logger.LogError(
        ex,
        "Could not publish biometric audit event {EventType} | IdT24: {IdT24}",
        auditEvent.Type.ToApiString(), auditEvent.IdT24);
}
}
}
}

```

7.9 Servicio de control de intentos

Archivo nuevo: `src/micro-person-api/Infrastructure/Biometric/BiometricAttemptControlService.cs`

Aquí vive toda la política: umbral, ventana, desbloqueo y qué se audita. Los handlers no saben nada de esto.

```

// Ruta destino:
// src/micro-person-api/Infrastructure/Biometric/BiometricAttemptControlService.cs
//
// La interfaz va en el mismo archivo por comodidad de lectura. Si el micro
// separa contratos, muévela a Common/Interfaces/Biometric/.
//
// Aquí vive TODA la política: umbral, ventana de bloqueo, desbloqueo automático
// y qué se audita. Los handlers no saben nada de esto; sólo reportan el
// resultado de su intento.

using Microsoft.Extensions.Logging;
using Microsoft.Extensions.Options;
using onboarding_micro_person.Common.Configuration;
using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Enums.Biometric;
using onboarding_micro_person.Common.Exceptions;
using onboarding_micro_person.Common.Interfaces.Biometric;
using onboarding_micro_person.Common.Models.Biometric;

namespace onboarding_micro_person.Infrastructure.Biometric
{
    public interface IBiometricAttemptControlService
    {
        /// <summary>
        /// Verifica el bloqueo ANTES de invocar a FacePhi. Si está bloqueado lanza
        /// BiometricAttemptsBlockedException (→ HTTP 423). Si el bloqueo ya expiró,
        /// lo libera y deja seguir.
        /// </summary>
        Task EnsureNotBlockedAsync(
            string idT24,
            BiometricFlow flow,
            CancellationToken cancellationToken);

        Task RegisterSuccessAsync(
            string idT24,
            BiometricFlow flow,
            BiometricAttemptContext context,
            CancellationToken cancellationToken);

        Task RegisterRejectionAsync(
            string idT24,
            BiometricFlow flow,
            BiometricAttemptContext context,
            CancellationToken cancellationToken);

        /// <summary>
        /// Error técnico: se audita pero NO incrementa el contador. Un cliente no
        /// puede quedar bloqueado porque FacePhi estuviera caído.
        /// </summary>

```

```

Task RegisterTechnicalErrorAsync(
    string idT24,
    BiometricFlow flow,
    string errorDetail,
    BiometricAttemptContext context,
    CancellationToken cancellationToken);
}

/// <summary>Trazabilidad de la petición. Ninguno de estos campos es dato biométrico.</summary>
public class BiometricAttemptContext
{
    public string? RequestId { get; set; }

    public string? OperationId { get; set; }

    public string? ServiceTransactionId { get; set; }

    public static readonly BiometricAttemptContext Empty = new();
}

public class BiometricAttemptControlService(
    IFacePhiAttemptControlRepository repository,
    IBiometricAuditPublisher auditPublisher,
    TimeProvider timeProvider,
    IOptions<AppSettings> settings,
    ILogger<BiometricAttemptControlService> logger) : IBiometricAttemptControlService
{
    private readonly IFacePhiAttemptControlRepository _repository = repository;
    private readonly IBiometricAuditPublisher _auditPublisher = auditPublisher;
    private readonly TimeProvider _timeProvider = timeProvider;
    private readonly AppSettings _settings = settings.Value;
    private readonly ILogger<BiometricAttemptControlService> _logger = logger;

    public async Task EnsureNotBlockedAsync(
        string idT24,
        BiometricFlow flow,
        CancellationToken cancellationToken)
    {
        if (!_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
        {
            return;
        }

        var utcNow = _timeProvider.GetUtcNow().UtcDateTime;
        var state = await _repository.GetByIdT24Async(idT24, cancellationToken);

        if (state is null || !state.IsBlocked)
        {
            return;
        }
    }
}

```

```

    }

    // Desbloqueo automático: no hace falta un job. El bloqueo sólo tiene
    // efecto en el momento en que alguien intenta validar, así que es ahí
    // donde se comprueba si la ventana ya venció. Un job programado
    // añadiría un componente más, dependería del reloj de cada pod y no
    // cambiaría nada de lo que ve el cliente.
    if (!state.IsCurrentlyBlocked(utcNow))
    {
        var released = await _repository.ReleaseExpiredBlockAsync(
            idT24, utcNow, cancellationToken);

        // released == null significa que otra petición simultánea ya lo
        // liberó. El resultado es el mismo: el cliente puede continuar.
        if (released is not null)
        {
            _logger.LogInformation(
                "Biometric block expired and was released | IdT24: {IdT24} | Flow: {Flow}",
                idT24, flow.ToApiString());

            await PublishAsync(
                BiometricAuditEventType AutomaticUnblock,
                idT24, flow, utcNow, "UNBLOCKED", released,
                BiometricAttemptContext.Empty, null, cancellationToken);
        }

        return;
    }

    var retryAfterSeconds = state.RemainingSeconds(utcNow);

    _logger.LogWarning(
        "Biometric validation attempted while blocked | IdT24: {IdT24} | Flow: {Flow} | " +
        "BlockedUntil: {BlockedUntil} | RetryAfterSeconds: {RetryAfterSeconds}",
        idT24, flow.ToApiString(), state.BlockedUntil, retryAfterSeconds);

    await PublishAsync(
        BiometricAuditEventType AttemptWhileBlocked,
        idT24, flow, utcNow, "BLOCKED", state,
        BiometricAttemptContext.Empty, null, cancellationToken);

    throw new BiometricAttemptsBlockedException(
        idT24, state.BlockedUntil!.Value, retryAfterSeconds);
}

public async Task RegisterSuccessAsync(
    string idT24,
    BiometricFlow flow,
    BiometricAttemptContext context,

```

```

CancellationToken cancellationToken)
{
    var utcNow = _timeProvider.GetUtcNow().UtcDateTime;

    if (!_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
    {
        // Con el control apagado se sigue auditando: se pierde el bloqueo,
        // no la trazabilidad.
        await PublishAsync(
            BiometricAuditEventType.AttemptSuccess,
            idT24, flow, utcNow, "APPROVED", null, context, null, cancellationToken);

        return;
    }

    var state = await _repository.RegisterSuccessfulAttemptAsync(
        idT24, flow, utcNow, cancellationToken);

    _logger.LogInformation(
        "Biometric attempt counter reset after success | IdT24: {IdT24} | Flow: {Flow}",
        idT24, flow.ToApiString());

    await PublishAsync(
        BiometricAuditEventType.AttemptSuccess,
        idT24, flow, utcNow, "APPROVED", state, context, null, cancellationToken);
}

public async Task RegisterRejectionAsync(
    string idT24,
    BiometricFlow flow,
    BiometricAttemptContext context,
    CancellationToken cancellationToken)
{
    var utcNow = _timeProvider.GetUtcNow().UtcDateTime;

    if (!_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
    {
        await PublishAsync(
            BiometricAuditEventType.AttemptFailed,
            idT24, flow, utcNow, "REJECTED", null, context, null, cancellationToken);

        return;
    }

    var state = await _repository.RegisterFailedAttemptAsync(
        idT24,
        flow,
        _settings.BIOMETRIC_MAX_FAILED_ATTEMPTS,
        _settings.BIOMETRIC_BLOCK_DURATION_MINUTES,

```

```

        utcNow,
        cancellationToken);

_logger.LogWarning(
    "Biometric attempt rejected | IdT24: {IdT24} | Flow: {Flow} | " +
    "FailedAttempts: {FailedAttempts} | Blocked: {Blocked}",
    idT24, flow.ToApiString(), state.FailedAttempts, state.IsBlocked);

await PublishAsync(
    BiometricAuditEventType.AttemptFailed,
    idT24, flow, utcNow, "REJECTED", state, context, null, cancellationToken);

// El bloqueo es un evento distinto del fallo que lo provoca: soporte
// necesita poder responder "¿cuándo se bloqueó?" sin recomponer la
// secuencia de fallos.
if (state.IsCurrentlyBlocked(utcNow))
{
    _logger.LogWarning(
        "Customer blocked for biometric validation | IdT24: {IdT24} | " +
        "FailedAttempts: {FailedAttempts} | BlockedUntil: {BlockedUntil}",
        idT24, state.FailedAttempts, state.BlockedUntil);

    await PublishAsync(
        BiometricAuditEventType.BlockedByExcess,
        idT24, flow, utcNow, "BLOCKED", state, context, null, cancellationToken);
}
}

public async Task RegisterTechnicalErrorAsync(
    string idT24,
    BiometricFlow flow,
    string errorDetail,
    BiometricAttemptContext context,
    CancellationToken cancellationToken)
{
    var utcNow = _timeProvider.GetUtcNow().UtcDateTime;

    // Deliberadamente NO se llama a RegisterFailedAttemptAsync.
    // Un error técnico no es un intento fallido del cliente.
    if (_settings.BIOMETRIC_ATTEMPT_CONTROL_ENABLED)
    {
        await _repository.TouchAttemptAsync(idT24, flow, utcNow, cancellationToken);
    }

    var state = await _repository.GetByIdT24Async(idT24, cancellationToken);

    _logger.LogError(
        "Biometric technical error, counter not incremented | IdT24: {IdT24} | " +
        "Flow: {Flow} | Detail: {Detail}",

```

```

        idT24, flow.ToApiString(), errorDetail);

    await PublishAsync(
        BiometricAuditEventType.TechnicalError,
        idT24, flow, utcNow, "ERROR", state, context, errorDetail, cancellationToken);
}

private Task PublishAsync(
    BiometricAuditEventType type,
    string idT24,
    BiometricFlow flow,
    DateTime utcNow,
    string result,
    FacePhiAttemptControlDocument? state,
    BiometricAttemptContext context,
    string? errorDetail,
    CancellationToken cancellationToken)
{
    var auditEvent = BiometricAuditEvent.For(type, idT24, flow, utcNow, result);

    auditEvent.FailedAttempts = state?.FailedAttempts ?? 0;
    auditEvent.Blocked = state?.IsCurrentlyBlocked(utcNow) ?? false;
    auditEvent.BlockedUntil = state?.BlockedUntil;
    auditEvent.RequestId = context.RequestId;
    auditEvent.OperationId = context.OperationId;
    auditEvent.ServiceTransactionId = context.ServiceTransactionId;
    auditEvent.ErrorDetail = errorDetail;

    return _auditPublisher.PublishAsync(auditEvent, cancellationToken);
}
}
}

```

7.10 Interfaz marcadora

Archivo nuevo: `src/micro-person-api/Common/Interfaces/Biometric/IBiometricAttemptControlled.cs`

```
// Ruta destino:
// src/micro-person-api/Common/Interfaces/Biometric/IBiometricAttemptControlled.cs
//
// Marca los Commands que consumen un intento biométrico. Sólo con implementar
// esta interfaz, un Command queda cubierto por el control de intentos: el
// behaviour se engancha por la restricción genérica, sin tocar el handler.
//
// Lo implementan ValidateIdentityV2Cmd y ValidateFaceOnlyCmd.
// NO lo implementa GetFacePhiDocumentQry: consultar si hay documento almacenado
// no invoca a FacePhi y por tanto no es un intento.

using onboarding_micro_person.Common.Enums.Biometric;

namespace onboarding_micro_person.Common.Interfaces.Biometric
{
    public interface IBiometricAttemptControlled
    {
        string IdT24 { get; }

        /// <summary>Flujo al que pertenece el intento. Se persiste como "último flujo ejecutado".</summary>
        BiometricFlow Flow { get; }

        /// <summary>operationId del tracking de FacePhi, si la petición lo trae. Sólo para trazabilidad.
        </summary>
        string? TrackingOperationId { get; }
    }
}
```

7.11 Behaviour del pipeline

Archivo nuevo: src/micro-person-api/Application/Common/Behaviours/BiometricAttemptControlBehaviour.cs


```

// Ruta destino:
// src/micro-person-api/Application/Common/Behaviours/BiometricAttemptControlBehaviour.cs
// (junto a ValidationBehaviour, PerformanceBehaviour y UnhandledExceptionBehaviour)
//
// Por qué un behaviour y no código dentro de cada handler:
// 1. El control es idéntico para validate y validate-face. Duplicarlo en dos
//     handlers garantiza que algún día uno de los dos se quede sin actualizar.
// 2. La regla "no se llama a FacePhi estando bloqueado" queda estructural: el
//     handler ni siquiera se ejecuta, no depende de que alguien recuerde poner
//     el return temprano en el sitio correcto.
// 3. Un tercer flujo biométrico futuro sólo tiene que implementar la interfaz.
//
// ORDEN DE REGISTRO (importante): después de ValidationBehaviour. Una petición
// con idT24 vacío debe morir como 400 de validación, no consumir un intento.

using FluentValidation;
using MediatR;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging;
using onboarding_micro_person.Application.Facephi.Commands;
using onboarding_micro_person.Common.Exceptions;
using onboarding_micro_person.Common.Interfaces.Biometric;
using onboarding_micro_person.Infrastructure.Biometric;

namespace onboarding_micro_person.Application.Common.Behaviours
{
    public class BiometricAttemptControlBehaviour<TRequest, TResponse>(
        IBiometricAttemptControlService attemptControl,
        IHttpContextAccessor httpContextAccessor,
        ILogger<BiometricAttemptControlBehaviour<TRequest, TResponse>> logger)
        : IPipelineBehavior<TRequest, TResponse>
        where TRequest : IBiometricAttemptControlled
    {
        private readonly IBiometricAttemptControlService _attemptControl = attemptControl;
        private readonly IHttpContextAccessor _httpContextAccessor = httpContextAccessor;
        private readonly ILogger<BiometricAttemptControlBehaviour<TRequest, TResponse>> _logger = logger;

        public async Task<TResponse> Handle(
            TRequest request,
            RequestHandlerDelegate<TResponse> next,
            CancellationToken cancellationToken)
        {
            // 1) Puerta de entrada. Si está bloqueado lanza 423 y next() nunca corre,
            //     así que no hay llamada a FacePhi.
            await _attemptControl.EnsureNotBlockedAsync(request.IdT24, request.Flow, cancellationToken);

            TResponse response;

            try

```

```

{
    response = await next();
}
catch (BiometricAttemptsBlockedException)
{
    // No debería llegar aquí (ya se comprobó arriba), pero si algún
    // handler la lanzara, pasa intacta: un bloqueo no es un fallo más.
    throw;
}
catch (ValidationException)
{
    // Petición mal formada: culpa del llamador, no intento del cliente.
    throw;
}
catch (NotFoundException)
{
    // "No hay documento almacenado" no es un rechazo biométrico:
    // FacePhi ni siquiera se invocó. No incrementa ni se audita como
    // error técnico.
    throw;
}
catch (Exception ex)
{
    // Cualquier otra excepción es error técnico: se audita, NO incrementa.
    await _attemptControl.RegisterTechnicalErrorAsync(
        request.IdT24,
        request.Flow,
        ex.GetType().Name,
        BuildContext(request, serviceTransactionId: null),
        cancellationToken);

    throw;
}

// 2) Resultado del intento. Se apoya en el tipo concreto que devuelven
// los dos handlers; si mañana aparece otro tipo de resultado,
// conviene extraer una interfaz IBiometricValidationOutcome.
if (response is ValidateIdentityV2Result outcome)
{
    var context = BuildContext(request, outcome.Data?.ServiceTransactionId);

    if (outcome.Status == FacialValidationStatus.Approved)
    {
        await _attemptControl.RegisterSuccessAsync(
            request.IdT24, request.Flow, context, cancellationToken);
    }
    else
    {
        await _attemptControl.RegisterRejectionAsync(

```

```

        request.IdT24, request.Flow, context, cancellationToken);
    }
}
else
{
    _logger.LogWarning(
        "Biometric attempt control could not interpret response of type {ResponseType} | IdT24:
{IdT24}",
        typeof(TResponse).Name, request.IdT24);
}

return response;
}

private BiometricAttemptContext BuildContext(TRequest request, string? serviceTransactionId) =>
    new()
    {
        RequestId = _httpContextAccessor.HttpContext?.Request.Headers["requestId"].FirstOrDefault(),
        OperationId = request.TrackingOperationId,
        ServiceTransactionId = serviceTransactionId
    };
}
}

```

7.12 Manejador del 423

Archivo nuevo: src/micro-person-api/Common/Exceptions/Handlers/BiometricAttemptsBlockedExceptionHandler.cs

```

// Ruta destino:
// src/micro-person-api/Common/Exceptions/Handlers/BiometricAttemptsBlockedExceptionHandler.cs
//
// El micro ya tiene manejo de excepciones (ExceptionHandler / middleware). Hay
// dos formas de integrar esto; usa la que corresponda a lo que encuentres:
//
// A) Si existe un CustomExceptionHandler con un diccionario de mapeos
//     (patrón de la plantilla Clean Architecture), añade la entrada:
//     { typeof(BiometricAttemptsBlockedException), HandleAttemptsBlockedException }
//     y copia el cuerpo de TryHandleAsync como método privado.
//
// B) Si no, registra este handler ANTES del genérico:
//     builder.Services.AddExceptionHandler<BiometricAttemptsBlockedExceptionHandler>();
//     (el orden importa: gana el primero que devuelve true).
//
// Lo que no es negociable es la salida: status 423, cabecera Retry-After y un
// cuerpo del que la app pueda sacar cuánto falta. Un 423 sin Retry-After obliga
// a Móvil a inventarse el temporizador.

using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;
using onboarding_micro_person.Common.Exceptions;

namespace onboarding_micro_person.Common.Exceptions.Handlers
{
    public class BiometricAttemptsBlockedExceptionHandler(
        ILogger<BiometricAttemptsBlockedExceptionHandler> logger) : IExceptionHandler
    {
        private const int StatusLocked = StatusCodes.Status423Locked;

        private readonly ILogger<BiometricAttemptsBlockedExceptionHandler> _logger = logger;

        public async ValueTask<bool> TryHandleAsync(
            HttpContext httpContext,
            Exception exception,
            CancellationToken cancellationToken)
        {
            if (exception is not BiometricAttemptsBlockedException blocked)
            {
                return false;
            }

            _logger.LogWarning(
                "Returning 423 Locked | IdT24: {IdT24} | BlockedUntil: {BlockedUntil} | RetryAfter: {RetryAfter}",
                blocked.IdT24, blocked.BlockedUntil, blocked.RetryAfterSeconds);

```

```

        httpContext.Response.StatusCode = StatusLocked;
        httpContext.Response.Headers.RetryAfter = blocked.RetryAfterSeconds.ToString();

        var problemDetails = new ProblemDetails
        {
            Status = StatusLocked,
            Title = "Biometric validation temporarily blocked",
            Detail = "The customer exceeded the allowed number of failed biometric attempts.",
            Type = "https://tools.ietf.org/html/rfc4918#section-11.3",
            Instance = httpContext.Request.Path
        };

        // Extensiones: lo que la app necesita para pintar la pantalla.
        // isValid viaja también aquí para que el cliente pueda leer siempre
        // el mismo campo, responda 200 o 423.
        problemDetails.Extensions["code"] = BiometricAttemptsBlockedException.ErrorCode;
        problemDetails.Extensions["isValid"] = false;
        problemDetails.Extensions["blocked"] = true;
        problemDetails.Extensions["blockedUntil"] = blocked.BlockedUntil;
        problemDetails.Extensions["retryAfterSeconds"] = blocked.RetryAfterSeconds;

        await httpContext.Response.WriteAsJsonAsync(problemDetails, cancellationToken);

        return true;
    }
}
}

```

7.13 Cambios en los dos comandos existentes

Cada comando declara su flujo. Son cuatro líneas por archivo y ningún cambio en los handlers.

`Application/Facephi/Commands/ValidateIdentityV2Cmd.cs`

```
// Añadir el using:
using onboarding_micro_person.Common.Enums.Biometric;
using onboarding_micro_person.Common.Interfaces.Biometric;

// Cambiar la declaración de la clase:
public class ValidateIdentityV2Cmd
    : IRequest<ValidateIdentityV2Result>, IBiometricAttemptControlled
{
    public string IdT24 { get; set; } = string.Empty;

    // ... el resto de propiedades se queda igual ...

    // Miembros de IBiometricAttemptControlled. No se serializan en la petición:
    // son metadatos que el behaviour lee del comando.
    [System.Text.Json.Serialization.JsonIgnore]
    public BiometricFlow Flow => BiometricFlow.ValidateBiometric;

    [System.Text.Json.Serialization.JsonIgnore]
    public string? TrackingOperationId => Tracking?.OperationId;
}
```

Application/Facephi/Commands/ValidateFaceOnlyCmd.cs

```
public class ValidateFaceOnlyCmd
    : IRequest<ValidateIdentityV2Result>, IBiometricAttemptControlled
{
    public string IdT24 { get; set; } = string.Empty;

    public string BestImageToken { get; set; } = string.Empty;

    public FacephiTrackingExtraData? Tracking { get; set; }

    [System.Text.Json.Serialization.JsonIgnore]
    public BiometricFlow Flow => BiometricFlow.ValidateFace;

    [System.Text.Json.Serialization.JsonIgnore]
    public string? TrackingOperationId => Tracking?.OperationId;
}
```

`GetFacePhiDocumentQry` **no** implementa la interfaz. Consultar si hay documento almacenado no invoca a FacePhi, luego no es un intento y no debe contar ni bloquearse.

7.14 Registro en `Program.cs` / `DependencyInjection.cs`

Sólo se añaden líneas; no se reemplaza nada.

```
// El behaviour lee el requestId de la cabecera para la trazabilidad de auditoría.
builder.Services.AddHttpContextAccessor();

// Reloj inyectable: permite probar la expiración del bloqueo sin esperar 20 minutos.
builder.Services.AddSingleton(TimeProvider.System);

// Persistencia del control de intentos.
builder.Services.AddScoped<IFacePhiAttemptControlRepository, FacePhiAttemptControlRepository>();
builder.Services.AddHostedService<FacePhiAttemptControlIndexInitializer>();

// Política y auditoría.
builder.Services.AddScoped<IBiometricAuditPublisher, BiometricAuditPublisher>();
builder.Services.AddScoped<IBiometricAttemptControlService, BiometricAttemptControlService>();

// Traducción del bloqueo a HTTP 423. Debe ir ANTES del manejador genérico:
// gana el primero que devuelve true.
builder.Services.AddExceptionHandler<BiometricAttemptsBlockedExceptionHandler>();
```

Y en el registro de MediatR, **después** de `ValidationBehaviour` :

```
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly());

    cfg.AddOpenBehavior(typeof(UnhandledExceptionBehaviour<,>));
    cfg.AddOpenBehavior(typeof(ValidationBehaviour<,>));

    // ← NUEVO, justo aquí: una petición con idT24 vacío debe morir como 400 de
    // validación, no consumir un intento del cliente.
    cfg.AddOpenBehavior(typeof(BiometricAttemptControlBehaviour<,>));

    cfg.AddOpenBehavior(typeof(PerformanceBehaviour<,>));
});
```

Sobre la restricción genérica: `BiometricAttemptControlBehaviour<TRequest, TResponse>` está restringido a `where TRequest : IBiometricAttemptControlled`. El contenedor de .NET 8 descarta automáticamente las instanciaciones que no cumplen la restricción, así que el behaviour sólo se aplica a los dos comandos biométricos y ninguna otra petición del micro lo atraviesa.

Si el proyecto registra los behaviours con `services.AddTransient(typeof(IPipelineBehavior<,>), ...)` en lugar de `cfg.AddOpenBehavior(...)`, el comportamiento es el mismo.

7.15 Configuración

`Common/Configuration/AppSettings.cs` — añadir tres propiedades:

```

/// <summary>Interruptor general del control de intentos. En false no se bloquea a nadie, pero se sigue
auditando.</summary>
public bool BIOMETRIC_ATTEMPT_CONTROL_ENABLED { get; set; } = true;

/// <summary>Fallos consecutivos que disparan el bloqueo.</summary>
public int BIOMETRIC_MAX_FAILED_ATTEMPTS { get; set; } = 5;

/// <summary>Duración del bloqueo temporal, en minutos.</summary>
public int BIOMETRIC_BLOCK_DURATION_MINUTES { get; set; } = 20;

```

appsettings.json — dentro de la sección **AppSettings** existente:

```

{
  "AppSettings": {
    "BIOMETRIC_ATTEMPT_CONTROL_ENABLED": true,
    "BIOMETRIC_MAX_FAILED_ATTEMPTS": 5,
    "BIOMETRIC_BLOCK_DURATION_MINUTES": 20
  }
}

```

ConfigMap de Kubernetes — mismos valores en los tres ambientes:

```

AppSettings__BIOMETRIC_ATTEMPT_CONTROL_ENABLED: "true"
AppSettings__BIOMETRIC_MAX_FAILED_ATTEMPTS: "5"
AppSettings__BIOMETRIC_BLOCK_DURATION_MINUTES: "20"

```

El interruptor no es decoración. `BIOMETRIC_ATTEMPT_CONTROL_ENABLED=false` desactiva el bloqueo **sin desplegar**, y la auditoría sigue funcionando. Si en producción aparece un falso positivo masivo —por ejemplo, una versión de la app que envía capturas de mala calidad y dispara rechazos legítimos en clientes reales— se apaga el bloqueo mientras se corrige, sin dejar de registrar lo que ocurre.

8. Implementación · **facade-security**

El requerimiento es explícito: *"No implementar ni duplicar la lógica de intentos"*. El facade sólo propaga.

8.1 **facephi.service.ts** — sin cambios

Ya funciona. `handleUpstreamError` reenvía el status del downstream tal cual:

```

const status = error.response?.status ?? HttpStatus.BAD_GATEWAY;
const errorPayload = error.response?.data ?? { ... };
throw new HttpException(errorPayload, status);

```

Un `423` del micro llega a axios como error (axios rechaza todo lo que no sea 2xx), entra por el `catch`, y sale como `HttpException(cuerpo, 423)`. El cuerpo viaja íntegro, así que `blockedUntil` y `retryAfterSeconds` llegan a la app. **No hay una sola línea que añadir.**

8.2 facephi.controller.ts — sólo Swagger

```
// facade-security · src/modules/facephi/facephi.controller.ts
//
// ÚNICO cambio necesario en el controller: documentar el 423 en los dos
// endpoints de validación. No se añade lógica: el requisito dice explícitamente
// "No implementar ni duplicar la lógica de intentos" en el facade.
//
// Añadir esta línea junto a los demás @ApiResponse de validateBiometric y de
// validateFaceOnly:

@ApiResponse({
  status: 423,
  description:
    'Validacion biometrica bloqueada temporalmente por exceso de intentos fallidos. ' +
    'El cuerpo incluye blockedUntil y retryAfterSeconds, y la cabecera Retry-After ' +
    'trae los segundos restantes.',
})

// Queda así en validateBiometric (extracto):
//
// @ApiOperation({ summary: 'Valida la fotografia del documento contra la captura facial' })
// @ApiResponse({ status: 200, description: 'isValid indica si la validacion biometrica fue aprobada o no' })
// @ApiResponse({ status: 400, description: 'Datos de entrada invalidos' })
// @ApiResponse({ status: 423, description: 'Validacion biometrica bloqueada temporalmente...' })
// @ApiResponse({ status: 502, description: 'Facephi no disponible' })
// @ApiResponse({ status: 504, description: 'Timeout invocando Facephi' })
// @HttpCode(HttpStatus.OK)
// @Post('validate-biometric')
//
// Y en validateFaceOnly, entre el 404 y el 502.
//
// Los endpoints de consulta de documento (GET document/:idT24 y POST document)
// NO llevan 423: no invocan a FacePhi, luego no son intentos y no se bloquean.
```

8.3 facephi.service.spec.ts — pruebas de propagación

Que hoy funcione sin cambios no significa que vaya a seguir funcionando. Estas pruebas existen para que, si alguien "normaliza" los errores del facade en el futuro, el 423 no se convierta silenciosamente en un 500 y la app pierda la pantalla de bloqueo.

```
// facade-security · src/modules/facephi/facephi.service.spec.ts
//
// Añadir estas pruebas dentro del describe('validateBiometric') existente.
// No hay cambio de código en facephi.service.ts: handleUpstreamError ya
// reenvía error.response.status tal cual, así que el 423 pasa sin tocar nada.
// Estas pruebas están para que siga siendo cierto: si mañana alguien "normaliza"
// los errores del facade, el 423 se convertiría en 500 y la app perdería la
// pantalla de bloqueo. Estas pruebas fallarían antes de que eso llegue a QA.

it('should propagate 423 when the customer is temporarily blocked', async () => {
  const blockedBody = {
    status: 423,
    title: 'Biometric validation temporarily blocked',
    code: 'BIOMETRIC_ATTEMPTS_BLOCKED',
    isValid: false,
    blocked: true,
    blockedUntil: '2026-08-25T10:20:00Z',
    retryAfterSeconds: 1200,
  };

  (httpService.axiosRef.post as jest.Mock).mockRejectedValue({
    isAxiosError: true,
    message: 'Request failed with status code 423',
    response: { status: 423, data: blockedBody },
  });

  await expect(
    service.validateBiometric(payload as any, headers as any),
  ).rejects.toMatchObject({ status: 423 });
});

it('should propagate the block payload untouched', async () => {
  // La app necesita blockedUntil y retryAfterSeconds para el temporizador.
  // Si el facade recorta el cuerpo, Movil tiene que inventarse la cuenta atras.
  const blockedBody = {
    code: 'BIOMETRIC_ATTEMPTS_BLOCKED',
    isValid: false,
    blocked: true,
    blockedUntil: '2026-08-25T10:20:00Z',
    retryAfterSeconds: 1200,
  };

  (httpService.axiosRef.post as jest.Mock).mockRejectedValue({
    isAxiosError: true,
    message: 'Request failed with status code 423',
    response: { status: 423, data: blockedBody },
  });

  await expect(
```

```

        service.validateBiometric(payload as any, headers as any),
    ).rejects.toMatchObject({ response: blockedBody });
});

// Y dentro del describe('validateFaceOnly'):

it('should propagate 423 on the face-only flow too', async () => {
    (httpService.axiosRef.post as jest.Mock).mockRejectedValue({
        isAxiosError: true,
        message: 'Request failed with status code 423',
        response: { status: 423, data: { code: 'BIOMETRIC_ATTEMPTS_BLOCKED' } },
    });

    await expect(
        service.validateFaceOnly({ idT24: '1' } as any, headers as any),
    ).rejects.toMatchObject({ status: 423 });
});

```

8.4 Verificación obligatoria antes de cerrar

Hay que confirmar que **nada entre el micro y la app remapea el 423**. Tres puntos a revisar:

Punto	Cómo verificar
Filtros/interceptores globales del facade	<code>grep -rn "ExceptionHandler\ @Catch\ useGlobalFilters" src/</code> — confirmar que ninguno convierte status desconocidos en 500
Middleware de cifrado del facade	Confirmar que cifra también los cuerpos de error, o que deja pasar el ProblemDetails legible. Es el mismo middleware que ya causó el problema del GET
API Gateway / APIM	Confirmar que 423 está en la lista de status permitidos y no se colapsa a 500

La forma rápida de cerrar los tres: hacer fallar cinco veces la validación en un ambiente de pruebas y **observar el status que recibe la app**, no el que emite el micro.

9. Catálogo de eventos de auditoría

9.1 Eventos

Evento	Cuándo se emite	result	¿Contador se mueve?
BIOMETRIC_ATTEMPT_SUCCESS	Validación aprobada	APPROVED	Se reinicia a 0
BIOMETRIC_ATTEMPT_FAILED	Validación rechazada por FacePhi	REJECTED	+1
BIOMETRIC_BLOCKED_BY_EXCESS	El fallo alcanzó el umbral	BLOCKED	No (lo movió el evento anterior)
BIOMETRIC_ATTEMPT_WHILE_BLOCKED	Llamada recibida durante el bloqueo	BLOCKED	No
BIOMETRIC_AUTOMATIC_UNBLOCK	La ventana expiró y se liberó	UNBLOCKED	Se reinicia a 0
BIOMETRIC_TECHNICAL_ERROR	Excepción técnica invocando FacePhi	ERROR	No

El bloqueo se emite como evento **separado** del fallo que lo provoca. Soporte necesita poder responder "¿cuándo se bloqueó este cliente?" consultando un evento, no recomponiendo la secuencia de cinco fallos.

9.2 Campos

Campo	Origen	Obligatorio
eventType	Catálogo de 9.1	Sí
idT24	Comando	Sí
occurredAt	TimeProvider, UTC	Sí
flow	validate-biometric validate-face	Sí
result	APPROVED REJECTED BLOCKED UNBLOCKED ERROR	Sí
failedAttempts	Estado tras aplicar el evento	Sí
blocked	Estado tras aplicar el evento	Sí
blockedUntil	Estado	Sólo si está bloqueado
requestId	Cabecera HTTP	Trazabilidad
operationId	tracking.operationId de la petición	Trazabilidad
serviceTransactionId	Respuesta de FacePhi	Trazabilidad
errorDetail	Tipo de excepción o código de servicio	Sólo en TECHNICAL_ERROR

9.3 Lo que nunca entra en la auditoría

token1 · token2 · bestImageToken · extraData · imágenes · stack traces completos.

Son datos biométricos o pueden contenerlos. La auditoría registra **el hecho**, no la evidencia. Hay una prueba automatizada que serializa todos los eventos generados y verifica que ninguna de esas cadenas aparece (AuditEvents_NeverCarryBiometricData); si alguien añade un campo de más, la prueba falla.

9.4 La auditoría no bloquea el flujo

Si `audit-log` está caído, el `BiometricAuditPublisher` registra un error en los logs del micro y el flujo continúa. Es una decisión consciente: la fuente de verdad del contador es Mongo, así que el control de intentos sigue siendo correcto aunque se pierdan eventos. La alternativa —fallar la validación biométrica de un cliente porque el servicio de auditoría no responde— cambia una pérdida de trazabilidad por una caída de un flujo crítico de cara al cliente.

Recomendación operativa: alertar sobre el log `Could not publish biometric audit event`. La pérdida de eventos tiene que ser detectable, no silenciosa.

10. El estado del Soft Token no se toca

El requerimiento lo dice expresamente: *"El estado del Soft Token debe permanecer sin cambios ante rechazos biométricos, errores técnicos o bloqueos."*

En este diseño se cumple **por construcción**: `onboarding-micro-person` no tiene ninguna dependencia hacia el dominio de Soft Token, y este desarrollo no le añade ninguna. El micro responde con un veredicto biométrico; la decisión de activar o no el token la toma quien orquesta el flujo de Soft Token, a partir de ese veredicto.

Lo que sí hay que garantizar es lo siguiente, y conviene dejarlo escrito para el equipo de Soft Token:

Respuesta del micro	Qué debe hacer el flujo de Soft Token
<code>200 { isValid: true }</code>	Continuar con la activación
<code>200 { isValid: false }</code>	No activar. No invalidar ni cambiar el estado del token
<code>423</code>	No activar. No contar como intento de token ni bloquear el token. Es un bloqueo biométrico, con su propio reloj
<code>502 / 504</code>	No activar. No cambiar nada. Permitir reintento

El riesgo real no está en el micro sino en la coordinación: si el flujo de Soft Token tiene su propio contador de intentos y también lo incrementa ante un 423, el cliente acabaría bloqueado dos veces por el mismo hecho. **Pregunta abierta para el equipo** (sección 16).

11. Bloqueo previo: distinguir rechazo de error técnico

Esta es la única parte del requerimiento que el código actual **no puede cumplir sin un cambio adicional**, y conviene que quede visible antes de estimar.

11.1 El problema

En `ValidateIdentityV2CmdHandler` y en `ValidateFaceOnlyCmdHandler`, cuando FacePhi responde con `serviceResultCode != 0` el código actual hace esto:

```

var approved = false;

if (result.ServiceResultCode != 0)
{
    _logger.LogWarning("Unsuccessful service result | ..."); // approved se queda en false
}
else
{
    // ... evaluación real ...
}

var validationStatus = approved
    ? FacialValidationStatus.Approved
    : FacialValidationStatus.Rejected; // ← un fallo del servicio sale como "Rejected"

```

Un fallo del servicio de FacePhi sale del handler indistinguible de un rechazo biométrico legítimo. Con el control de intentos activo, eso significa que **una caída de FacePhi bloquea a todos los clientes que lo intenten cinco veces** — precisamente lo que el requerimiento prohíbe al pedir que "un error técnico no incremente el contador".

11.2 Solución recomendada

Cuando `serviceResultCode != 0`, lanzar una excepción de integración en lugar de devolver `Rejected`:

```

if (result.ServiceResultCode != 0)
{
    _logger.LogError(
        "Unsuccessful service result | ServiceResultCode: {ServiceResultCode} | " +
        "TransactionId: {TransactionId} | IdT24: {IdT24}",
        result.ServiceResultCode, result.ServiceTransactionId, cmd.IdT24);

    throw new FacePhiIntegrationException(
        $"FacePhi returned service result code {result.ServiceResultCode}.");
}

```

El behaviour la reconoce como error técnico, la audita, **no toca el contador**, y el manejador de excepciones la traduce a `502 Bad Gateway`.

Esto encaja con dos correcciones que ya estaban pendientes de este desarrollo y que conviene cerrar en el mismo despliegue:

Pendiente	Estado actual	Debe quedar
<code>ValidationException</code> sin manejador	Devuelve <code>500</code>	<code>400</code>
<code>EnsureSuccessStatusCode()</code> sobre respuesta de FacePhi	<code>HttpRequestException</code> → <code>500</code>	<code>FacePhiIntegrationException</code> → <code>502</code> , y <code>TaskCanceledException</code> → <code>504</code>
<code>serviceResultCode != 0</code>	<code>200 { isValid: false }</code>	<code>502</code>

11.3 Impacto en la app y alternativa

El cambio tiene consecuencia visible: donde hoy la app recibe `200 { isValid: false }` ante una caída de FacePhi, pasará a recibir `502`. Eso **es lo correcto** —decirle al cliente "no eres tú" cuando el problema es del proveedor es un error de producto— pero hay que avisar a Móvil APAP antes de desplegarlo.

Si por calendario la app no puede manejar el `502` todavía, la alternativa de menor fricción es marcar el resultado sin cambiar el status:

```
// Variante de transición: la app sigue recibiendo 200 { isValid: false },
// pero el control de intentos sabe que no fue culpa del cliente.
public class ValidateIdentityV2Result
{
    public FacialValidationStatus Status { get; init; }
    public bool IsTechnicalFailure { get; init; } // ← campo nuevo, no se expone al cliente
    // ...
}
```

y en el behaviour, comprobar `IsTechnicalFailure` antes de decidir entre éxito y rechazo. Es peor semánticamente, pero cumple el requerimiento del contador sin romper la app.

Recomendación: ir por la 11.2 y coordinar con Móvil. La variante de transición deja una deuda que después nadie quita.

12. Pruebas

12.1 Los nueve escenarios exigidos

#	Escenario del requerimiento	Prueba
1	Un fallo incrementa el contador	<code>RegisterRejection_IncrementsCounter</code>
2	El cuarto fallo no bloquea	<code>FourthRejection_DoesNotBlock</code>
3	El quinto fallo bloquea	<code>FifthRejection_BlocksForConfiguredDuration</code>
4	No se invoca a FacePhi durante el bloqueo	<code>WhenBlocked_HandlerIsNeverInvoked</code>
5	Un éxito reinicia el contador	<code>SuccessfulAttempt_ResetsCounter</code>
6	Un error técnico no incrementa	<code>TechnicalError_DoesNotIncrementCounter</code> · <code>RepeatedTechnicalErrors_NeverBlockTheCustomer</code>
7	El bloqueo expira a los 20 minutos	<code>BlockExpires_AfterConfiguredMinutes</code>
8	Los eventos se registran en auditoría	<code>EveryEvent_IsAudited</code>
9	El facade propaga el 423	<code>should propagate 423 when the customer is temporarily blocked</code>

12.2 Escenarios añadidos

Prueba	Qué protege
<code>AttemptWhileBlocked_DoesNotIncrementCounterFurther</code>	Que martillear el endpoint no alargue el bloqueo indefinidamente
<code>AfterUnblock_CustomerGetsFullSetOfAttemptsAgain</code>	Que tras el desbloqueo el cliente recupere los 5 intentos, no que quede a uno del siguiente bloqueo
<code>FlowsShareTheSameCounter</code>	Que alternar <code>validate</code> y <code>validate-face</code> no duplique los intentos disponibles
<code>Counter_IsPerCustomer</code>	Que bloquear a un cliente no afecte a los demás
<code>WhenNotFound_DoesNotTouchTheCounter</code>	Que "sin documento almacenado" no gaste un intento
<code>WhenValidationFails_DoesNotTouchTheCounter</code>	Que una petición mal formada no gaste un intento
<code>WhenDisabled_NothingIsBlockedButEverythingIsAudited</code>	Que el interruptor de emergencia funcione y no se lleve la auditoría por delante
<code>AuditEvents_NeverCarryBiometricData</code>	Que nadie añada <code>token1</code> al evento de auditoría
<code>should propagate the block payload untouched</code>	Que el facade no recorte <code>blockedUntil</code> / <code>retryAfterSeconds</code>

12.3 Código de las pruebas

Archivo nuevo: `tests/micro-person-api.Tests/Application/Biometric/BiometricAttemptControlTests.cs`

La política se prueba contra un **repositorio en memoria** que reproduce la semántica de Mongo, no contra un mock. Con un mock, "el quinto fallo bloquea" sólo probaría que el servicio llama al método correcto; con el repositorio en memoria se prueba la secuencia completa de intentos.


```
// Ruta destino:
// tests/micro-person-api.Tests/Application/Biometric/BiometricAttemptControlTests.cs
//
// Estrategia: la política (umbral, bloqueo, reinicio, expiración) se prueba
// contra un repositorio en memoria que reproduce la semántica de Mongo, no
// contra un mock. Con un mock, "el quinto fallo bloquea" sólo probaría que el
// servicio llama al método correcto; con el repositorio en memoria se prueba de
// verdad la secuencia completa de intentos.
//
// Lo que este archivo NO cubre y necesita prueba de integración con Mongo real
// (Testcontainers o Mongo2Go): que $inc y el filtro condicional se comporten de
// forma atómica bajo concurrencia, y que el índice único sobre idT24 impida
// documentos duplicados. Ver la sección de pruebas del documento.

using FluentValidation;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging.Abstractions;
using Microsoft.Extensions.Options;
using Moq;
using NUnit.Framework;
using onboarding_micro_person.Application.Common.Behaviours;
using onboarding_micro_person.Application.Facephi.Commands;
using onboarding_micro_person.Common.Configuration;
using onboarding_micro_person.Common.Entities;
using onboarding_micro_person.Common.Enums.Biometric;
using onboarding_micro_person.Common.Exceptions;
using onboarding_micro_person.Common.Interfaces.Biometric;
using onboarding_micro_person.Common.Models.Biometric;
using onboarding_micro_person.Infrastructure.Biometric;

namespace onboarding_micro_person.Tests.Application.Biometric
{
    // -----
    // Dobles de prueba
    // -----

    /// <summary>Reloj controlable. TimeProvider es de la BCL en .NET 8, no requiere paquete.</summary>
    public class TestTimeProvider : TimeProvider
    {
        public DateTimeOffset.UtcNow { get; set; } = new(2026, 8, 25, 10, 0, 0, TimeSpan.Zero);

        public override DateTimeOffset GetUtcNow() => UtcNow;

        public void Advance(TimeSpan by) => UtcNow = UtcNow.Add(by);
    }

    /// <summary>Reproduce la semántica de FacePhiAttemptControlRepository sin Mongo.</summary>
    public class InMemoryAttemptControlRepository : IFacePhiAttemptControlRepository
    {

```

```

private readonly Dictionary<string, FacePhiAttemptControlDocument> _store = new();

public Task<FacePhiAttemptControlDocument?> GetByIdT24Async(
    string idT24, CancellationToken cancellationToken)
{
    _store.TryGetValue(idT24, out var document);

    return Task.FromResult(Clone(document));
}

public Task<FacePhiAttemptControlDocument> RegisterFailedAttemptAsync(
    string idT24, BiometricFlow flow, int maxFailedAttempts,
    int blockDurationMinutes, DateTime utcNow, CancellationToken cancellationToken)
{
    var document = GetOrCreate(idT24, utcNow);

    document.FailedAttempts += 1;
    document.LastFlow = flow.ToApiString();
    document.LastAttemptAt = utcNow;
    document.UpdatedAt = utcNow;

    if (document.FailedAttempts >= maxFailedAttempts && !document.IsBlocked)
    {
        document.IsBlocked = true;
        document.BlockedAt = utcNow;
        document.BlockedUntil = utcNow.AddMinutes(blockDurationMinutes);
    }

    return Task.FromResult(Clone(document)!);
}

public Task<FacePhiAttemptControlDocument> RegisterSuccessfulAttemptAsync(
    string idT24, BiometricFlow flow, DateTime utcNow, CancellationToken cancellationToken)
{
    var document = GetOrCreate(idT24, utcNow);

    document.FailedAttempts = 0;
    document.IsBlocked = false;
    document.BlockedAt = null;
    document.BlockedUntil = null;
    document.LastFlow = flow.ToApiString();
    document.LastAttemptAt = utcNow;
    document.UpdatedAt = utcNow;

    return Task.FromResult(Clone(document)!);
}

public Task<FacePhiAttemptControlDocument?> ReleaseExpiredBlockAsync(
    string idT24, DateTime utcNow, CancellationToken cancellationToken)

```

```

{
    if (!_store.TryGetValue(idT24, out var document)
        || !document.IsBlocked
        || document.BlockedUntil > utcNow)
    {
        return Task.FromResult<FacePhiAttemptControlDocument?>(null);
    }

    document.IsBlocked = false;
    document.FailedAttempts = 0;
    document.BlockedAt = null;
    document.BlockedUntil = null;
    document.UpdatedAt = utcNow;

    return Task.FromResult(Clone(document));
}

public Task TouchAttemptAsync(
    string idT24, BiometricFlow flow, DateTime utcNow, CancellationToken cancellationToken)
{
    var document = GetOrCreate(idT24, utcNow);

    document.LastFlow = flow.ToApiString();
    document.LastAttemptAt = utcNow;
    document.UpdatedAt = utcNow;

    return Task.CompletedTask;
}

private FacePhiAttemptControlDocument GetOrCreate(string idT24, DateTime utcNow)
{
    if (!_store.TryGetValue(idT24, out var document))
    {
        document = new FacePhiAttemptControlDocument
        {
            IdT24 = idT24,
            CreatedAt = utcNow,
            UpdatedAt = utcNow
        };

        _store[idT24] = document;
    }

    return document;
}

private static FacePhiAttemptControlDocument? Clone(FacePhiAttemptControlDocument? source)
{
    if (source is null)

```

```

        {
            return null;
        }

        return new FacePhiAttemptControlDocument
        {
            IdT24 = source.IdT24,
            FailedAttempts = source.FailedAttempts,
            IsBlocked = source.IsBlocked,
            BlockedAt = source.BlockedAt,
            BlockedUntil = source.BlockedUntil,
            LastFlow = source.LastFlow,
            LastAttemptAt = source.LastAttemptAt,
            CreatedAt = source.CreatedAt,
            UpdatedAt = source.UpdatedAt
        };
    }
}

public class RecordingAuditPublisher : IBiometricAuditPublisher
{
    public List<BiometricAuditEvent> Events { get; } = new();

    public Task PublishAsync(BiometricAuditEvent auditEvent, CancellationToken cancellationToken)
    {
        Events.Add(auditEvent);

        return Task.CompletedTask;
    }

    public int CountOf(BiometricAuditEventType type) => Events.Count(e => e.Type == type);
}

// -----
// Política de intentos
// -----

[TestFixture]
public class BiometricAttemptControlServiceTests
{
    private const string IdT24 = "123456789";
    private const int MaxAttempts = 5;
    private const int BlockMinutes = 20;

    private InMemoryAttemptControlRepository _repository = null!;
    private RecordingAuditPublisher _audit = null!;
    private TestTimeProvider _timeProvider = null!;
    private BiometricAttemptControlService _service = null!;

```

```

[SetUp]
public void Setup()
{
    _repository = new InMemoryAttemptControlRepository();
    _audit = new RecordingAuditPublisher();
    _timeProvider = new TestTimeProvider();
    _service = BuildService(enabled: true);
}

private BiometricAttemptControlService BuildService(bool enabled)
{
    var settings = new AppSettings
    {
        BIOMETRIC_ATTEMPT_CONTROL_ENABLED = enabled,
        BIOMETRIC_MAX_FAILED_ATTEMPTS = MaxAttempts,
        BIOMETRIC_BLOCK_DURATION_MINUTES = BlockMinutes
    };

    return new BiometricAttemptControlService(
        _repository,
        _audit,
        _timeProvider,
        Options.Create(settings),
        NullLogger<BiometricAttemptControlService>.Instance);
}

private Task RejectAsync() => _service.RegisterRejectionAsync(
    IdT24, BiometricFlow.ValidateFace, BiometricAttemptContext.Empty, CancellationToken.None);

private Task ApproveAsync() => _service.RegisterSuccessAsync(
    IdT24, BiometricFlow.ValidateFace, BiometricAttemptContext.Empty, CancellationToken.None);

private Task GuardAsync() => _service.EnsureNotBlockedAsync(
    IdT24, BiometricFlow.ValidateFace, CancellationToken.None);

private async Task<FacePhiAttemptControlDocument?> StateAsync() =>
    await _repository.GetByIdT24Async(IdT24, CancellationToken.None);

[Test]
public async Task RegisterRejection_IncrementsCounter()
{
    await RejectAsync();

    var state = await StateAsync();

    Assert.That(state, Is.Not.Null);
    Assert.That(state!.FailedAttempts, Is.EqualTo(1));
    Assert.That(state.IsBlocked, Is.False);
    Assert.That(state.LastFlow, Is.EqualTo("validate-face"));
}

```

```

        Assert.That(state.LastAttemptAt, Is.EqualTo(_timeProvider.UtcNow.UtcDateTime));
    }

    [Test]
    public async Task FourthRejection_DoesNotBlock()
    {
        for (var i = 0; i < 4; i++)
        {
            await RejectAsync();
        }

        var state = await StateAsync();

        Assert.That(state!.FailedAttempts, Is.EqualTo(4));
        Assert.That(state.IsBlocked, Is.False);

        // Y sigue pudiendo intentar.
        Assert.DoesNotThrowAsync(GuardAsync);
    }

    [Test]
    public async Task FifthRejection_BlocksForConfiguredDuration()
    {
        for (var i = 0; i < 5; i++)
        {
            await RejectAsync();
        }

        var state = await StateAsync();

        Assert.That(state!.FailedAttempts, Is.EqualTo(5));
        Assert.That(state.IsBlocked, Is.True);
        Assert.That(
            state.BlockedUntil,
            Is.EqualTo(_timeProvider.UtcNow.UtcDateTime.AddMinutes(BlockMinutes)));

        Assert.That(_audit.CountOf(BiometricAuditEventType.BlockedByExcess), Is.EqualTo(1));
    }

    [Test]
    public async Task AttemptWhileBlocked_Throws423AndAudits()
    {
        for (var i = 0; i < 5; i++)
        {
            await RejectAsync();
        }

        var exception = Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync);
    }

```

```

        Assert.That(exception!.IdT24, Is.EqualTo(IdT24));
        Assert.That(exception.RetryAfterSeconds, Is.EqualTo(BlockMinutes * 60));
        Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptWhileBlocked), Is.EqualTo(1));
    }

[Test]
public async Task AttemptWhileBlocked_DoesNotIncrementCounterFurther()
{
    // El intento bloqueado no llega a FacePhi, así que tampoco cuenta:
    // de lo contrario, martillear el endpoint alargaría el bloqueo del
    // cliente indefinidamente.
    for (var i = 0; i < 5; i++)
    {
        await RejectAsync();
    }

    Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync);
    Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync);

    var state = await StateAsync();

    Assert.That(state!.FailedAttempts, Is.EqualTo(5));
    Assert.That(
        state.BlockedUntil,
        Is.EqualTo(_timeProvider.UtcNow.UtcDateTime.AddMinutes(BlockMinutes)),
        "el reloj del bloqueo no se reinicia con cada intento rechazado");
}

[Test]
public async Task SuccessfulAttempt_ResetsCounter()
{
    await RejectAsync();
    await RejectAsync();
    await RejectAsync();

    await ApproveAsync();

    var state = await StateAsync();

    Assert.That(state!.FailedAttempts, Is.EqualTo(0));
    Assert.That(state.IsBlocked, Is.False);
    Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptSuccess), Is.EqualTo(1));
}

[Test]
public async Task TechnicalError_DoesNotIncrementCounter()
{
    await RejectAsync();

```

```

        await _service.RegisterTechnicalErrorAsync(
            IdT24,
            BiometricFlow.ValidateFace,
            "FacephiIntegrationException",
            BiometricAttemptContext.Empty,
            CancellationToken.None);

        var state = await StateAsync();

        Assert.That(state!.FailedAttempts, Is.EqualTo(1), "un error técnico no es un intento fallido del
cliente");
        Assert.That(_audit.CountOf(BiometricAuditEventType.TechnicalError), Is.EqualTo(1));
    }

    [Test]
    public async Task RepeatedTechnicalErrors_NeverBlockTheCustomer()
    {
        for (var i = 0; i < 20; i++)
        {
            await _service.RegisterTechnicalErrorAsync(
                IdT24, BiometricFlow.ValidateBiometric, "HttpRequestException",
                BiometricAttemptContext.Empty, CancellationToken.None);
        }

        var state = await StateAsync();

        Assert.That(state!.FailedAttempts, Is.EqualTo(0));
        Assert.That(state.IsBlocked, Is.False);
        Assert.DoesNotThrowAsync(GuardAsync);
    }

    [Test]
    public async Task BlockExpires_AfterConfiguredMinutes()
    {
        for (var i = 0; i < 5; i++)
        {
            await RejectAsync();
        }

        _timeProvider.Advance(TimeSpan.FromMinutes(BlockMinutes - 1));
        Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync, "a los 19 minutos sigue
bloqueado");

        _timeProvider.Advance(TimeSpan.FromMinutes(2));
        Assert.DoesNotThrowAsync(GuardAsync, "a los 21 minutos ya puede intentar");

        var state = await StateAsync();

        Assert.That(state!.IsBlocked, Is.False);
    }

```



```

        Assert.That(state.FailedAttempts, Is.EqualTo(0), "al expirar el bloqueo el contador vuelve a cero");
        Assert.That(_audit.CountOf(BiometricAuditEventType.AutomaticUnblock), Is.EqualTo(1));
    }

[Test]
public async Task AfterUnblock_CustomerGetsFullSetOfAttemptsAgain()
{
    for (var i = 0; i < 5; i++)
    {
        await RejectAsync();
    }

    _timeProvider.Advance(TimeSpan.FromMinutes(BlockMinutes + 1));
    await GuardAsync();

    for (var i = 0; i < 4; i++)
    {
        await RejectAsync();
    }

    var state = await StateAsync();

    Assert.That(state!.IsBlocked, Is.False, "cuatro fallos tras el desbloqueo no vuelven a bloquear");
}

[Test]
public async Task EveryEvent_IsAudited()
{
    for (var i = 0; i < 5; i++)
    {
        await RejectAsync();
    }

    Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync);

    _timeProvider.Advance(TimeSpan.FromMinutes(BlockMinutes + 1));
    await GuardAsync();
    await ApproveAsync();

    Assert.Multiple(() =>
    {
        Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptFailed), Is.EqualTo(5));
        Assert.That(_audit.CountOf(BiometricAuditEventType.BlockedByExcess), Is.EqualTo(1));
        Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptWhileBlocked), Is.EqualTo(1));
        Assert.That(_audit.CountOf(BiometricAuditEventType.AutomaticUnblock), Is.EqualTo(1));
        Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptSuccess), Is.EqualTo(1));
    });
}

```

```

[Test]
public async Task AuditEvents_NeverCarryBiometricData()
{
    await RejectAsync();

    var serialized = System.Text.Json.JsonSerializer.Serialize(_audit.Events);

    Assert.Multiple(() =>
    {
        Assert.That(serialized, Does.Not.Contain("token1"));
        Assert.That(serialized, Does.Not.Contain("token2"));
        Assert.That(serialized, Does.Not.Contain("bestImageToken"));
        Assert.That(serialized, Does.Not.Contain("extraData"));
    });
}

[Test]
public async Task WhenDisabled_NothingIsBlockedButEverythingIsAudited()
{
    _service = BuildService(enabled: false);

    for (var i = 0; i < 10; i++)
    {
        await RejectAsync();
    }

    Assert.DoesNotThrowAsync(GuardAsync);
    Assert.That(await StateAsync(), Is.Null, "con el control apagado no se escribe estado");
    Assert.That(_audit.CountOf(BiometricAuditEventType.AttemptFailed), Is.EqualTo(10));
}

[Test]
public async Task Counter_IsPerCustomer()
{
    for (var i = 0; i < 5; i++)
    {
        await RejectAsync();
    }

    Assert.DoesNotThrowAsync(() => _service.EnsureNotBlockedAsync(
        "987654321", BiometricFlow.ValidateFace, CancellationToken.None));
}

[Test]
public async Task FlowsShareTheSameCounter()
{
    // Cinco fallos repartidos entre los dos endpoints bloquean igual: el
    // control es por cliente, no por endpoint. Si no, bastaría con
    // alternar validate y validate-face para duplicar los intentos.

```

```

        for (var i = 0; i < 3; i++)
        {
            await _service.RegisterRejectionAsync(
                IdT24, BiometricFlow.ValidateBiometric, BiometricAttemptContext.Empty,
CancellationTokens.None);
        }

        for (var i = 0; i < 2; i++)
        {
            await _service.RegisterRejectionAsync(
                IdT24, BiometricFlow.ValidateFace, BiometricAttemptContext.Empty, CancellationTokens.None);
        }

        Assert.ThrowsAsync<BiometricAttemptsBlockedException>(GuardAsync);

        var state = await StateAsync();

        Assert.That(state!.LastFlow, Is.EqualTo("validate-face"));
    }
}

// -----
// Behaviour: garantiza que FacePhi no se invoca estando bloqueado
// -----

[TestFixture]
public class BiometricAttemptControlBehaviourTests
{
    private const string IdT24 = "123456789";

    private Mock<IBiometricAttemptControlService> _attemptControl = null!;
    private BiometricAttemptControlBehaviour<ValidateFaceOnlyCmd, ValidateIdentityV2Result> _behaviour =
null!;

    [SetUp]
    public void SetUp()
    {
        _attemptControl = new Mock<IBiometricAttemptControlService>();

        var httpContextAccessor = new HttpContextAccessor
        {
            HttpContext = new DefaultHttpContext()
        };

        httpContextAccessor.HttpContext!.Request.Headers["requestId"] = "req-1";

        _behaviour = new BiometricAttemptControlBehaviour<ValidateFaceOnlyCmd, ValidateIdentityV2Result>(
            _attemptControl.Object,
            httpContextAccessor,

```

```

        NullLogger<BiometricAttemptControlBehaviour<ValidateFaceOnlyCmd,
ValidateIdentityV2Result>>.Instance);
    }

    private static ValidateFaceOnlyCmd Command() => new()
    {
        IdT24 = IdT24,
        BestImageToken = "best-image-token"
    };

    // Ajusta la construcción del resultado a la firma real de
    // ValidateIdentityV2Result en el micro.
    private static ValidateIdentityV2Result Result(FacialValidationStatus status) =>
        new(status, null!);

    [Test]
    public async Task WhenBlocked_HandlerIsNeverInvoked()
    {
        _attemptControl
            .Setup(x => x.EnsureNotBlockedAsync(IdT24, BiometricFlow.ValidateFace,
It.IsAny<CancellationToken>()))
            .ThrowsAsync(new BiometricAttemptsBlockedException(IdT24, DateTime.UtcNow.AddMinutes(20),
1200));

        var handlerInvoked = false;

        Assert.ThrowsAsync<BiometricAttemptsBlockedException>(() => _behaviour.Handle(
            Command(),
            () =>
            {
                handlerInvoked = true;

                return Task.FromResult(Result(FacialValidationStatus.Approved));
            },
            CancellationToken.None));

        Assert.That(handlerInvoked, Is.False, "estando bloqueado no puede haber llamada a FacePhi");

        _attemptControl.Verify(
            x => x.RegisterRejectionAsync(
                It.IsAny<string>(), It.IsAny<BiometricFlow>(),
                It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
            Times.Never);

        await Task.CompletedTask;
    }

    [Test]
    public async Task WhenApproved_RegistersSuccess()

```

```

{
    await _behaviour.Handle(
        Command(),
        () => Task.FromResult(Result(FacialValidationStatus.Approved)),
        CancellationToken.None);

    _attemptControl.Verify(
        x => x.RegisterSuccessAsync(
            IdT24, BiometricFlow.ValidateFace,
            It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
        Times.Once);
}

[Test]
public async Task WhenRejected_RegistersRejection()
{
    await _behaviour.Handle(
        Command(),
        () => Task.FromResult(Result(FacialValidationStatus.Rejected)),
        CancellationToken.None);

    _attemptControl.Verify(
        x => x.RegisterRejectionAsync(
            IdT24, BiometricFlow.ValidateFace,
            It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
        Times.Once);
}

// FacePhiIntegrationException es la excepción del prerequisite descrito en
// la sección 11 del documento. Si en el micro se llama de otra forma,
// ajusta el nombre aquí y en el behaviour.
[Test]
public void WhenTechnicalException_RegistersTechnicalErrorAndRethrows()
{
    Assert.ThrowsAsync<FacePhiIntegrationException>(() => _behaviour.Handle(
        Command(),
        () => throw new FacePhiIntegrationException("FacePhi unavailable"),
        CancellationToken.None));

    _attemptControl.Verify(
        x => x.RegisterTechnicalErrorAsync(
            IdT24, BiometricFlow.ValidateFace, "FacePhiIntegrationException",
            It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
        Times.Once);

    _attemptControl.Verify(
        x => x.RegisterRejectionAsync(
            It.IsAny<string>(), It.IsAny<BiometricFlow>(),
            It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),

```

```

        Times.Never());
    }

    [Test]
    public void WhenNotFound_DoesNotTouchTheCounter()
    {
        // El cliente sin documento almacenado no gastó un intento: FacePhi
        // nunca se invocó.
        Assert.ThrowsAsync<NotFoundException>(() => _behaviour.Handle(
            Command(),
            () => throw new NotFoundException("Customer does not have a stored document."),
            CancellationToken.None));

        _attemptControl.Verify(
            x => x.RegisterRejectionAsync(
                It.IsAny<string>(), It.IsAny<BiometricFlow>(),
                It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
            Times.Never);

        _attemptControl.Verify(
            x => x.RegisterTechnicalErrorAsync(
                It.IsAny<string>(), It.IsAny<BiometricFlow>(), It.IsAny<string>(),
                It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
            Times.Never);
    }

    [Test]
    public void WhenValidationFails_DoesNotTouchTheCounter()
    {
        Assert.ThrowsAsync<ValidationException>(() => _behaviour.Handle(
            Command(),
            () => throw new ValidationException("idT24 is required."),
            CancellationToken.None));

        _attemptControl.Verify(
            x => x.RegisterRejectionAsync(
                It.IsAny<string>(), It.IsAny<BiometricFlow>(),
                It.IsAny<BiometricAttemptContext>(), It.IsAny<CancellationToken>()),
            Times.Never);
    }
}
}

```

12.4 Lo que estas pruebas no cubren

Dos cosas necesitan **prueba de integración contra un Mongo real** (Testcontainers o Mongo2Go):

1. **Atomicidad bajo concurrencia.** Que `$inc` y el filtro condicional se comporten correctamente con 20 peticiones simultáneas. Prueba sugerida: lanzar 20 rechazos en paralelo para el mismo `idT24` y verificar que `failedAttempts == 20` exactamente y que sólo hay **un** `blockedAt`.

2. **Índice único.** Que dos upserts concurrentes sobre un `idT24` inexistente no creen dos documentos.

12.5 Prueba manual end-to-end

```
# 1) Cinco rechazos consecutivos (usar una selfie que no corresponda al documento)
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w "intento $i -> %{http_code}\n" \
    -X POST http://localhost:5000/api/v1/facephi/validate-face \
    -H "Content-Type: application/json" \
    -H "requestId: manual-$i" \
    -d '{"idT24": "999999999", "bestImageToken": "<token>"}'
done
# esperado: 200, 200, 200, 200, 200    (los cinco con isValid:false)

# 2) Sexto intento -> bloqueado
curl -i -X POST http://localhost:5000/api/v1/facephi/validate-face \
  -H "Content-Type: application/json" \
  -d '{"idT24": "999999999", "bestImageToken": "<token>"}'
# esperado: HTTP/1.1 423 Locked + Retry-After: ~1200
```

```
// 3) Verificar el estado persistido
db.facephi_attempt_control.findOne({ idT24: "999999999" })
// esperado: failedAttempts: 5, isBlocked: true, blockedUntil ≈ ahora + 20 min

// 4) Forzar la expiración sin esperar 20 minutos
db.facephi_attempt_control.updateOne(
  { idT24: "999999999" },
  { $set: { blockedUntil: new Date(Date.now() - 1000) } }
)
// repetir el paso 2 -> debe devolver 200 y dejar failedAttempts en 0
```

Para el paso 5 (verificar que un error técnico no incrementa) sirve el stub local de FacePhi ya construido: apuntar el micro al stub y usar un `token1` que dispare la respuesta `http502`. El contador debe quedarse donde estaba.

13. Riesgos

Riesgo	Impacto	Mitigación
Una caída de FacePhi bloquea clientes masivamente	Alto	Los errores técnicos no incrementan el contador. Requiere el cambio de la sección 11 — es un prerequisite, no una mejora
Un algo remapea el 423 en el camino (filtro del facade, cifrado, APIM)	Alto: la app nunca ve el bloqueo y muestra un error genérico	Verificación de la sección 8.4, con prueba end-to-end observando el status que recibe la app
Falta el índice único → contadores duplicados	Alto: el control se puede burlar	<code>FacePhiAttemptControlIndexInitializer</code> , más el índice pendiente en <code>facephi_biometrics</code>
Carrera: dos peticiones simultáneas pasan la puerta	Bajo: en el peor caso 6 llamadas a FacePhi en lugar de 5	Aceptado y documentado. El <code>\$inc</code> atómico garantiza que el contador queda correcto; el bloqueo se aplica igual, sólo que un intento más tarde. Cerrarlo del todo exigiría reservar el intento antes de llamar a FacePhi, lo que penalizaría a los clientes cuya petición falle por red
audit-log caído	Medio: pérdida de trazabilidad	El flujo continúa y el error queda en logs. Alertar sobre <code>Could not publish biometric audit event</code>
Rechazos legítimos masivos (versión de la app con capturas de mala calidad)	Alto: clientes reales bloqueados	<code>BIOMETRIC_ATTEMPT_CONTROL_ENABLED=false</code> desactiva el bloqueo sin desplegar, conservando la auditoría
El flujo de Soft Token tiene su propio contador y también reacciona al 423	Medio: doble bloqueo por el mismo hecho	Coordinar con el equipo de Soft Token (sección 10 y pregunta abierta)
Sin canal de desbloqueo manual	Medio: soporte no puede ayudar a un cliente legítimo bloqueado	Con 20 minutos de ventana puede ser aceptable. Si no lo es, hace falta un endpoint administrativo (fuera del alcance actual, ver preguntas abiertas)

14. Plan de trabajo

#	Subtarea	Alcance	Est.
1	Prerrequisito: distinguir error técnico	FacePhiIntegrationException , mapeo 400/502/504, <code>serviceResultCode != 0</code> → excepción	3
2	Persistencia del estado	Entidad, repositorio con operaciones atómicas, inicializador de índices	3
3	Política de control	BiometricAttemptControlService , excepción de bloqueo, manejador 423	3
4	Enganche en el pipeline	Behaviour, interfaz marcadora, cambios en los dos comandos, registros en DI	2
5	Auditoría	Modelo de evento, publicador, integración con el cliente real de <code>audit-log</code>	2
6	Facade y pruebas	Swagger 423, pruebas de propagación, verificación end-to-end del status	2
		Total	15

Orden de ejecución: 1 → 2 → 3 → 4 → 5 → 6. La subtarea 1 es un prerrequisito real: sin ella, la 3 no puede cumplir la regla de los errores técnicos.

Las subtareas 2, 3 y 4 se pueden desplegar con `BIOMETRIC_ATTEMPT_CONTROL_ENABLED=false` para validar en ambiente sin afectar a nadie, y encender el control cuando la 5 y la 6 estén listas.

15. Checklist de descubrimiento en el repositorio

Antes de escribir la primera línea, resolver estas seis búsquedas. El código de este documento asume nombres que hay que confirmar.

Qué	Comando	Qué se hace con el resultado
Cliente de audit-log	<code>grep -rn "IAuditLog\ AuditService\ AuditClient\ audit-log" --include=*.cs src/</code>	Sustituir <code>IAuditLogService.RegisterAsync</code> en <code>BiometricAuditPublisher</code> por la firma real
Cómo se resuelve IMongoDatabase	<code>grep -rn "IMongoDatabase\ IMongoClient" --include=*.cs src/</code>	Replicar el patrón en <code>FacePhiAttemptControlRepository</code>
Manejo de excepciones existente	<code>grep -rn "IExceptionHandler\ ProblemDetails\ ExceptionMiddleware" --include=*.cs src/</code>	Decidir entre la opción A y la B de la sección 7.12
Registro de behaviours de MediatR	<code>grep -rn "AddOpenBehavior\ IPipelineBehavior" --include=*.cs src/</code>	Insertar el nuevo behaviour en la posición correcta
¿Ya hay un inicializador de índices?	<code>grep -rn "CreateIndex\ Indexes.CreateOne" --include=*.cs src/</code>	Añadir ahí los índices en lugar de crear una clase nueva
¿Ya hay abstracción de reloj?	<code>grep -rn "IDateTimeProvider\ IClock\ TimeProvider" --include=*.cs src/</code>	Usar la existente en lugar de <code>TimeProvider</code>

16. Preguntas abiertas

Cinco decisiones que no dependen del backend y conviene cerrar antes de codificar.

1. **¿Se le dicen al cliente los intentos restantes?** El requerimiento no lo pide. Exponer "te quedan 2 intentos" es mejor UX y ligeramente peor desde seguridad (le confirma al atacante cuánto margen tiene). El dato está disponible; es decisión de producto y seguridad, no técnica.
2. **¿El contador tiene ventana temporal?** Tal como está escrito el requerimiento, tres fallos de hace seis meses siguen contando para el bloqueo de hoy. El campo `lastAttemptAt` ya se persiste, así que añadir un "reiniciar si el último intento fue hace más de X minutos" no requeriría migración. **¿Es el comportamiento deseado o los fallos deben caducar?**
3. **¿Hace falta desbloqueo manual desde soporte?** Con 20 minutos puede que no. Si un cliente legítimo bloqueado necesita atención inmediata, haría falta un endpoint administrativo con su propia autenticación y auditoría — está fuera del alcance actual y sería una subtarea aparte.
4. **¿El flujo de Soft Token lleva su propio contador de intentos?** Si lo lleva y también reacciona al 423, el cliente quedaría bloqueado dos veces por el mismo hecho, con dos relojes distintos. Hay que alinearlos con el equipo de Soft Token.
5. **¿Puede Móvil APAP manejar el 502 de la sección 11.3?** De la respuesta depende si se implementa la solución recomendada o la variante de transición.

17. Resumen de archivos

onboarding-micro-person

Archivo	Acción
Common/Entities/FacePhiAttemptControlDocument.cs	Nuevo
Common/Enums/Biometric/BiometricFlow.cs	Nuevo
Common/Models/Biometric/BiometricAuditEvent.cs	Nuevo
Common/Interfaces/Biometric/IFacePhiAttemptControlRepository.cs	Nuevo
Common/Interfaces/Biometric/IBiometricAttemptControlled.cs	Nuevo
Common/Exceptions/BiometricAttemptsBlockedException.cs	Nuevo
Common/Exceptions/Handlers/BiometricAttemptsBlockedExceptionHandler.cs	Nuevo
Persistence/FacePhiAttemptControlRepository.cs	Nuevo
Persistence/FacePhiAttemptControlIndexInitializer.cs	Nuevo
Infrastructure/Biometric/BiometricAttemptControlService.cs	Nuevo
Infrastructure/Biometric/BiometricAuditPublisher.cs	Nuevo
Application/Common/Behaviours/BiometricAttemptControlBehaviour.cs	Nuevo
Application/Facephi/Commands/ValidateIdentityV2Cmd.cs	Modificado — implementa la interfaz marcadora
Application/Facephi/Commands/ValidateFaceOnlyCmd.cs	Modificado — implementa la interfaz marcadora + prerequisite §11
Common/Configuration/AppSettings.cs	Modificado — 3 variables
Program.cs / DependencyInjection.cs	Modificado — sólo se añaden líneas
appsettings.json + ConfigMaps	Modificado — 3 variables por ambiente
tests/.../BiometricAttemptControlTests.cs	Nuevo — 21 pruebas

facade-security

Archivo	Acción
src/modules/facephi/facephi.controller.ts	Modificado — sólo @ApiResponse({ status: 423 })
src/modules/facephi/facephi.service.ts	Sin cambios
src/modules/facephi/facephi.service.spec.ts	Modificado — 3 pruebas de propagación del 423

Nueva infraestructura

Recurso	Detalle
Colección Mongo	facephi_attempt_control en la base de datos que ya usa el micro
Índices	ux_idT24 (único) · ix_isBlocked_blockedUntil
Variables de entorno	BIOMETRIC_ATTEMPT_CONTROL_ENABLED · BIOMETRIC_MAX_FAILED_ATTEMPTS · BIOMETRIC_BLOCK_DURATION_MINUTES